

# Curso Profesional de Entity Framework Core 8

## Módulo 2 - Modelado de datos con Entity Framework Core

TEORÍA

AUTOR: JAIME GALLO

.NET 8 · Entity Framework Core 8 · Visual Studio Community · SQL Server LocalDB · Proyecto AceriaData

# Índice

---

|                                                                                          |    |
|------------------------------------------------------------------------------------------|----|
| Punto 2.1 – Convenciones de modelado en Entity Framework Core                            | 3  |
| Punto 2.2 – Entidades y propiedades: tipos, requeridos, longitudes y valores por defecto | 8  |
| Punto 2.3 – Relaciones uno a muchos                                                      | 13 |
| Punto 2.4 – Relaciones uno a uno                                                         | 18 |
| Punto 2.5 – Relaciones muchos a muchos                                                   | 24 |
| Punto 2.6 – Configuración mediante Data Annotations                                      | 29 |
| Punto 2.7 – Configuración mediante Fluent API                                            | 37 |
| Punto 2.8 – Claves primarias, claves alternativas y claves compuestas                    | 41 |
| Punto 2.9 – Índices y restricciones                                                      | 46 |
| Punto 2.10 – Filtros globales de consulta                                                | 50 |
| Punto 2.11 – Soft Delete: implementación, consultas y restauración                       | 51 |
| Punto 2.12 – Integración de EF Core en Clean Architecture y Arquitectura Hexagonal       | 54 |

# Módulo 2 - Modelado de datos con Entity Framework Core 8

El Módulo 2 continúa AceriaData desde el estado final del Módulo 1. El objetivo es comprender cómo EF Core construye y configura el modelo, cómo expresa relaciones y restricciones y cómo ese modelo evoluciona hasta una organización por capas en la que EF Core queda aislado en infraestructura.

## Punto 2.1 - Convenciones de modelado en Entity Framework Core

**Audiencia:** Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

**Proyecto:** Se analizan las convenciones que EF Core aplica al modelo actual del proyecto AceriaData y se prepara el terreno para configurar el modelo explícitamente en los siguientes puntos.

### Objetivos de aprendizaje

Comprender qué son las convenciones de modelado y por qué existen.

Identificar las convenciones de nombre de tabla y columna.

Reconocer las convenciones de clave primaria y clave foránea.

Entender las convenciones de nulabilidad y de tipos de datos.

Identificar las convenciones de relaciones uno a muchos, uno a uno y muchos a muchos.

Aplicar estas convenciones al proyecto AceriaData y observar el modelo resultante.

### Teoría

#### Qué son las convenciones de modelado

Las convenciones de modelado son un conjunto de reglas que EF Core aplica automáticamente para construir el modelo a partir de las clases de entidad. Cuando se crea una instancia del DbContext, EF Core inspecciona las propiedades DbSet<T>, las clases referenciadas y sus propiedades, y deduce cómo mapearlas a tablas, columnas, claves y relaciones. Estas reglas permiten que el modelo funcione sin configuración explícita, siempre que las clases sigan ciertas pautas.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public double Ancho { get; set; }
    public double Largo { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

public class AceriaDbContext : DbContext
{
    public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;
    public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;

    public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }
}
```

CSHARP

Con este código, sin ninguna configuración adicional, EF Core deduce que existe una tabla `OrdenesFabricacion` con columnas `Id`, `NumeroOrden`, `Cliente` y `FechaCreacion`, y una tabla `PlanchasAcero` con columnas `Id`, `OrdenId`, `Espesor`, `Ancho` y `Largo`. También deduce que `PlanchaAcero` tiene una clave foránea `OrdenId` que apunta a `OrdenFabricacion`. Todo ello sin haber escrito una sola línea de configuración.

Las convenciones no son mágicas: son reglas documentadas que EF Core aplica en un orden concreto. Conocerlas permite saber cuándo se puede confiar en ellas y cuándo es necesario configurar el modelo explícitamente.

## Convención de nombre de tabla

EF Core deduce el nombre de la tabla a partir del nombre de la propiedad `DbSet<T>`. Si la propiedad se llama `OrdenesFabricacion`, la tabla se llama `OrdenesFabricacion`. Si la propiedad se llama `PlanchasAcero`, la tabla se llama `PlanchasAcero`.

```
public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;  
public DbSet<PlanchaAcero> PlanchasAcero { get; set; } = null!;  
public DbSet<Aleacion> Aleaciones { get; set; } = null!;  
public DbSet<EstadoOrden> EstadosOrden { get; set; } = null!;
```

CSHARP

La primera línea crea la tabla `OrdenesFabricacion`. La segunda crea la tabla `PlanchasAcero`. La tercera crea la tabla `Aleaciones`. La cuarta crea la tabla `EstadosOrden`. El nombre de la tabla coincide exactamente con el nombre de la propiedad.

Si no existiera una propiedad `DbSet<T>` para una entidad, EF Core usaría el nombre de la clase. Por ejemplo, si `EstadoOrden` no tuviera un `DbSet`, la tabla se llamaría `EstadoOrden` en lugar de `EstadosOrden`.

## Convención de nombre de columna

EF Core deduce el nombre de la columna a partir del nombre de la propiedad de la entidad. Si la propiedad se llama `NumeroOrden`, la columna se llama `NumeroOrden`. Si la propiedad se llama `FechaCreacion`, la columna se llama `FechaCreacion`.

```
public class OrdenFabricacion  
{  
    public int Id { get; set; }  
    public string NumeroOrden { get; set; } = string.Empty;  
    public string Cliente { get; set; } = string.Empty;  
    public DateTime FechaCreacion { get; set; }  
}
```

CSHARP

La primera línea declara la propiedad `Id`, que se mapea a la columna `Id`. La segunda declara la propiedad `NumeroOrden`, que se mapea a la columna `NumeroOrden`. La tercera declara la propiedad `Cliente`, que se mapea a la columna `Cliente`. La cuarta declara la propiedad `FechaCreacion`, que se mapea a la columna `FechaCreacion`.

Esta convención es útil porque mantiene la coherencia entre el código y la base de datos. Sin embargo, cuando la base de datos ya existe y usa nombres distintos, es necesario configurar el mapeo explícitamente.

## Convención de clave primaria

EF Core reconoce una clave primaria por convención cuando una propiedad se llama `Id` o `<NombreDelTipo>Id`, sin distinguir mayúsculas y minúsculas. El nombre de la propiedad `DbSet` no forma parte de esta convención.

```
public class OrdenFabricacion  
{  
    public int Id { get; set; }  
    public string NumeroOrden { get; set; } = string.Empty;  
}  
  
public class PlanchaAcero  
{  
    public int PlanchaAceroId { get; set; }  
    public double Espesor { get; set; }  
}
```

CSHARP

En `OrdenFabricacion`, `Id` se reconoce como clave primaria. En `PlanchaAcero`, `PlanchaAceroId` también se reconoce porque sigue el patrón `<NombreDelTipo>Id`. Una propiedad `OrdenId` dentro de `OrdenFabricacion` no sería clave primaria por esta convención; ese nombre suele corresponder a una clave foránea cuando existe una navegación `Orden`.

Las claves se estudian de forma explícita en el punto 2.8, donde se usan `HasKey`, `HasAlternateKey` y claves compuestas.

## Convención de clave foránea

EF Core detecta claves foráneas por convención cuando una propiedad de navegación apunta a otra entidad y existe una propiedad escalar cuyo nombre sigue el patrón <NombreDeNavegacion>Id o <NombreDeEntidadPrincipal>Id.

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea declara la propiedad Id, que es la clave primaria de PlanchaAcero. La segunda línea declara la propiedad OrdenId, que EF Core detecta como clave foránea porque sigue el patrón <NombreDeNavegacion>Id. La tercera línea declara la propiedad Orden, que es la propiedad de navegación. EF Core relaciona OrdenId con Orden y crea la clave foránea correspondiente en la tabla PlanchasAcero.

Si la propiedad se llamara OrdenFabricacionId, EF Core también la detectaría como clave foránea. Si se llamara OrdenId, también. La regla es que el nombre de la propiedad contenga el nombre de la propiedad de navegación o el nombre de la entidad principal, seguido de Id.

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenFabricacionId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea declara la propiedad Id. La segunda línea declara la propiedad OrdenFabricacionId, que EF Core detecta como clave foránea porque contiene el nombre de la entidad principal seguido de Id. La tercera línea declara la propiedad de navegación.

## Convención de nulabilidad

EF Core deduce la nulabilidad de una columna a partir del tipo de la propiedad. Si la propiedad es de un tipo de referencia no anulable (string, por ejemplo), la columna se crea como NOT NULL. Si la propiedad es de un tipo de referencia anulable (string?), la columna se crea como NULL. Si la propiedad es de un tipo de valor anulable (int?, DateTime?), la columna se crea como NULL.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string? Observaciones { get; set; }
    public DateTime FechaCreacion { get; set; }
    public DateTime? FechaEntrega { get; set; }
}
```

CSHARP

La primera línea declara Id de tipo int, que se mapea a una columna NOT NULL. La segunda línea declara NumeroOrden de tipo string no anulable, que se mapea a una columna NOT NULL. La tercera línea declara Observaciones de tipo string? anulable, que se mapea a una columna NULL. La cuarta línea declara FechaCreacion de tipo DateTime no anulable, que se mapea a una columna NOT NULL. La quinta línea declara FechaEntrega de tipo DateTime? anulable, que se mapea a una columna NULL.

Esta convención depende de que la opción Nullable esté habilitada en el archivo .csproj. Si no lo está, todas las propiedades de tipo referencia se consideran anulables por defecto y las columnas se crean como NULL.

## Convención de tipo de dato

EF Core mapea los tipos de C# a tipos de SQL Server por convención. int se mapea a int, long a bigint, string a nvarchar(max), bool a bit, DateTime a datetime2, decimal a decimal(18,2), double a float y Guid a uniqueidentifier.

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public double Espesor { get; set; }
    public decimal Peso { get; set; }
    public stringCodigo { get; set; } = string.Empty;
    public DateTime FechaFabricacion { get; set; }
}
```

CSHARP

```
public bool Activa { get; set; }
}
```

La primera línea declara Id de tipo int, que se mapea a int. La segunda línea declara Espesor de tipo double, que se mapea a float. La tercera línea declara Peso de tipo decimal, que se mapea a decimal(18,2). La cuarta línea declara Codigo de tipo string, que se mapea a nvarchar(max). La quinta línea declara FechaFabricacion de tipo DateTime, que se mapea a datetime2. La sexta línea declara Activa de tipo bool, que se mapea a bit.

El mapeo de string a nvarchar(max) es uno de los más importantes. Si no se configura una longitud máxima, la columna se crea sin límite, lo que puede afectar al rendimiento y a la indexación. En el Punto 2.2 se configurará la longitud máxima para las propiedades de tipo string.

## Convención de relación uno a muchos

EF Core detecta una relación uno a muchos cuando una entidad tiene una propiedad de navegación de colección y la entidad relacionada tiene una propiedad de navegación de referencia y una clave foránea.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea de OrdenFabricacion declara la colección Planchas, que es el extremo "muchos" de la relación. La primera línea de PlanchaAcero declara OrdenId, que es la clave foránea. La segunda línea de PlanchaAcero declara Orden, que es el extremo "uno" de la relación. EF Core detecta la relación y la configura automáticamente con DeleteBehavior.Cascade por convención.

## Convención de relación uno a uno

EF Core detecta una relación uno a uno cuando ambas entidades tienen una propiedad de navegación de referencia y una de ellas tiene una clave foránea que también es clave primaria, o cuando la clave foránea es única.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public DetalleOrden? Detalle { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea de OrdenFabricacion declara Detalle, que es una navegación de referencia. En DetalleOrden, OrdenId puede actuar como clave foránea por convención porque su nombre corresponde al tipo principal OrdenFabricacion abreviado mediante la navegación Orden. Cuando EF Core determina una relación uno a uno con DetalleOrden como dependiente, la clave foránea queda protegida mediante unicidad en el modelo relacional. En AceriaData la relación se configura explícitamente en el punto 2.4 para evitar ambigüedades sobre el dependiente y la obligatoriedad.

## Convención de relación muchos a muchos

EF Core detecta una relación muchos a muchos cuando ambas entidades tienen una propiedad de navegación de colección y no existe una entidad intermedia explícita. En este caso, EF Core crea automáticamente una tabla intermedia con las claves foráneas de ambas entidades.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public List<Aleacion> Aleaciones { get; set; } = new();
}
```

CSHARP

```
public class Aleacion
{
    public int Id { get; set; }
    public List<OrdenFabricacion> Ordenes { get; set; } = new();
}
```

La primera línea de OrdenFabricacion declara la colección Aleaciones y la primera línea de Aleacion declara la colección Ordenes. EF Core puede descubrir por convención una relación muchos a muchos y crear una entidad de unión implícita que se mapea a una tabla de unión. Los nombres concretos de la tabla y de sus columnas se generan por convención a partir de los tipos y las navegaciones, por lo que conviene inspeccionar el modelo o la migración en lugar de codificar una dependencia sobre un nombre supuesto. En 2.5 AceriaData usa una entidad intermedia explícita porque la relación necesita almacenar datos propios.

Cómo inspeccionar el modelo

EF Core permite inspeccionar el modelo construido a través de la propiedad Model del DbContext. Esta propiedad expone las entidades, sus propiedades, sus claves y sus relaciones.

```
foreach (var entidad in context.Model.GetEntityTypes())
{
    Console.WriteLine($"Entidad: {entidad.ClrType.Name} → Tabla: {entidad.GetTableName()}");

    foreach (var propiedad in entidad.GetProperties())
    {
        Console.WriteLine($" Propiedad: {propiedad.Name} | Columna: {propiedad.GetColumnName()} | Tipo: {propiedad.ClrType.Name} | Nullable: {propiedad.IsNullable}");
    }

    foreach (var claveForanea in entidad.GetForeignKeys())
    {
        Console.WriteLine($" FK: {claveForanea.PrincipalEntityType.ClrType.Name} → {claveForanea.Properties.First().Name}");
    }
}
```

CSHARP

La primera línea itera sobre cada entidad del modelo. La segunda muestra el nombre de la clase y el nombre de la tabla. La tercera itera sobre cada propiedad de la entidad. La cuarta muestra el nombre de la propiedad, la columna, el tipo y si es anulable. La quinta itera sobre cada clave foránea. La sexta muestra la entidad principal y el nombre de la propiedad de la clave foránea.

Cuándo es necesario configurar el modelo explícitamente

Las convenciones cubren la mayoría de los casos, pero hay situaciones en las que es necesario configurar el modelo explícitamente. La primera es cuando los nombres de las tablas o columnas no coinciden con los nombres de las propiedades. La segunda es cuando la clave primaria no sigue el patrón Id. La tercera es cuando la clave foránea no sigue el patrón <Navegacion>Id. La cuarta es cuando se quiere cambiar el comportamiento de eliminación. La quinta es cuando se quiere añadir índices, restricciones o valores por defecto.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("Ordenes");
        entity.HasKey(o => o.Id);
        entity.Property(o => o.NumeroOrden).HasMaxLength(50).IsRequired();
    });
}
```

CSHARP

La primera línea declara el método OnModelCreating. La segunda selecciona la entidad OrdenFabricacion. La tercera cambia el nombre de la tabla a Ordenes. La cuarta declara la clave primaria. La quinta configura la propiedad NumeroOrden con longitud máxima de 50 caracteres y la marca como requerida.

Esta configuración explícita se estudiará en detalle en los puntos 2.6 y 2.7.

## El proyecto AceriaData

En el proyecto AceriaData, el modelo actual se apoya por completo en las convenciones. Las entidades OrdenFabricacion, PlanchaAcero, Aleacion y EstadoOrden se mapean a tablas con los mismos nombres que sus propiedades DbSet. Las claves primarias se llaman Id. Las claves foráneas siguen el patrón <Navegacion>Id. Las relaciones se detectan automáticamente. En este punto se inspecciona el modelo construido y se identifican las convenciones aplicadas.

## Resumen de la teoría

Las convenciones son reglas que EF Core aplica automáticamente para construir el modelo.

El nombre de la tabla se deduce del nombre de la propiedad DbSet.

El nombre de la columna se deduce del nombre de la propiedad de la entidad.

La clave primaria se detecta por el nombre Id o <Clase>Id.

La clave foránea se detecta por el nombre <Navegacion>Id o <Entidad>Id.

La nulabilidad se deduce del tipo de la propiedad.

El tipo de dato se deduce del tipo de C#.

Las relaciones uno a muchos, uno a uno y muchos a muchos se detectan por las propiedades de navegación.

La propiedad Model permite inspeccionar el modelo construido.

Cuando las convenciones no son suficientes, se configura el modelo explícitamente.

## Punto 2.2 - Entidades y propiedades: tipos, requeridos, longitudes y valores por defecto

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configuran explícitamente las propiedades de las entidades del proyecto AceriaData para adaptar el modelo a las necesidades reales de la acería, definiendo longitudes máximas, propiedades requeridas, precisión decimal y valores por defecto.

### Objetivos de aprendizaje

Comprender cómo se configuran las propiedades de las entidades en EF Core.

Diferenciar entre configuración por convención, por Data Annotations y por Fluent API.

Configurar longitudes máximas para propiedades de tipo string.

Configurar propiedades requeridas y opcionales.

Configurar precisión decimal para propiedades de tipo decimal.

Configurar valores por defecto para propiedades escalares.

Aplicar estas configuraciones al proyecto AceriaData.

### Teoría

Por qué configurar las propiedades explícitamente

Las convenciones de EF Core cubren la mayoría de los casos, pero no todos. Cuando una propiedad de tipo string se mapea a nvarchar(max), la columna no tiene límite de longitud. Esto puede afectar al rendimiento, impedir la creación de índices y permitir datos más largos de lo esperado. Cuando una propiedad de tipo decimal se mapea a decimal(18,2), la precisión y la escala son fijas y pueden no coincidir con las necesidades del dominio. Cuando una propiedad es opcional, la columna se crea como NULL, lo que puede no ser deseable.

La configuración explícita permite adaptar el modelo a las reglas del dominio. En una acería, el número de orden tiene un formato concreto y una longitud máxima. El nombre del cliente tiene una longitud limitada. El peso de una plancha tiene una precisión decimal concreta. Estas reglas se reflejan en el modelo para que la base de datos las aplique.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
    public DateTime FechaCreacion { get; set; }
}
```

CSHARP

Con las convenciones por defecto, NumeroOrden y Cliente se mapean a nvarchar(max). Sin embargo, el número de orden en una acería tiene un formato como OF-2024-0001, que no supera los veinte caracteres. El nombre del cliente rara vez supera los doscientos caracteres. Configurar estas longitudes mejora el rendimiento y evita datos inconsistentes.

Las tres formas de configurar el modelo

EF Core permite configurar el modelo de tres formas: por convención, por Data Annotations y por Fluent API. Las convenciones son las reglas automáticas que ya se estudiaron en el punto anterior. Las Data Annotations son atributos que se aplican directamente sobre las propiedades de las entidades. La Fluent API es una configuración imperativa que se escribe en el método OnModelCreating del DbContext.

```
// Data Annotations
public class OrdenFabricacion
{
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
}

// Fluent API
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>()
        .Property(o => o.NumeroOrden)
        .IsRequired()
        .HasMaxLength(50);
}
```

CSHARP

La primera forma aplica atributos directamente sobre la propiedad. La segunda forma configura la propiedad desde el método OnModelCreating. Ambas producen el mismo resultado. La Fluent API tiene prioridad sobre las Data Annotations, y las Data Annotations tienen prioridad sobre las convenciones.

## Configurar propiedades requeridas

Una propiedad requerida se mapea a una columna NOT NULL. Por convención, las propiedades de tipo valor no anulable (int, DateTime, bool) son requeridas, y las propiedades de tipo referencia no anulable (string sin ?) también lo son cuando la opción Nullable está habilitada. Para marcar explícitamente una propiedad como requerida, se usa el atributo [Required] o el método IsRequired.

```
[Required]
public string NumeroOrden { get; set; } = string.Empty;
```

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.NumeroOrden)
    .IsRequired();
```

CSHARP

La primera forma aplica el atributo sobre la propiedad. La segunda configura la propiedad desde OnModelCreating. Ambas generan una columna NOT NULL.

## Configurar propiedades opcionales

Una propiedad opcional se mapea a una columna NULL. Por convención, las propiedades de tipo valor anulable (int?, DateTime?) y las propiedades de tipo referencia anulable (string?) son opcionales. Para marcar explícitamente una propiedad como opcional, se usa el método IsRequired(false).

```
public string? Observaciones { get; set; }
```

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.Observaciones)
    .IsRequired(false);
```

CSHARP

La primera forma declara la propiedad como anulable. La segunda configura la propiedad desde OnModelCreating. Ambas generan una columna NULL.

## Configurar longitudes máximas

La longitud máxima de una propiedad de tipo string se configura con el atributo [MaxLength] o con el método HasMaxLength. Si no se configura, la columna se crea como nvarchar(max), que no tiene límite de longitud.

```
[MaxLength(50)]
public string NumeroOrden { get; set; } = string.Empty;

[MaxLength(200)]
public string Cliente { get; set; } = string.Empty;

[MaxLength(500)]
public string? Observaciones { get; set; }
```

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.NumeroOrden).HasMaxLength(50);
    entity.Property(o => o.Cliente).HasMaxLength(200);
    entity.Property(o => o.Observaciones).HasMaxLength(500);
});
```

CSHARP

La primera forma aplica los atributos sobre las propiedades. La segunda configura las propiedades desde OnModelCreating. Ambas generan columnas nvarchar(50), nvarchar(200) y nvarchar(500) respectivamente.

## Configurar precisión decimal

La precisión y la escala de una propiedad de tipo decimal se configuran con el atributo [Precision] o con el método HasPrecision. La precisión es el número total de dígitos, y la escala es el número de dígitos después del punto decimal.

```
[Precision(18, 2)]
public decimal Peso { get; set; }
```

CSHARP

```
modelBuilder.Entity<PlanchaAcero>()
    .Property(p => p.Peso)
    .HasPrecision(18, 2);
```

CSHARP

La primera forma aplica el atributo sobre la propiedad. La segunda configura la propiedad desde OnModelCreating. Ambas generan una columna decimal(18,2).

La precisión por defecto para decimal en SQL Server es decimal(18,2). Si se necesita más precisión, como para el peso de una plancha con tres decimales, se configura HasPrecision(18, 3).

## Configurar valores por defecto

El valor por defecto de una propiedad se configura con el método HasDefaultValue o HasDefaultValueSql. El primero establece un valor constante, y el segundo una expresión SQL que se evalúa en el servidor.

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.FechaCreacion)
    .HasDefaultValueSql("GETDATE()");

modelBuilder.Entity<PlanchaAcero>()
    .Property(p => p.Activa)
    .HasDefaultValue(true);

modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.Estado)
    .HasDefaultValue("Pendiente");
```

CSHARP

La primera configuración establece GETDATE() como valor por defecto para FechaCreacion. La segunda establece true como valor por defecto para Activa. La tercera establece "Pendiente" como valor por defecto para Estado.

El valor por defecto se aplica cuando la propiedad no se asigna explícitamente en el INSERT. Si la propiedad se asigna, el valor asignado prevalece.

## Configurar el tipo de columna

El tipo de columna se configura con el método HasColumnType. Es útil cuando se quiere usar un tipo específico del motor que no coincide con el mapeo por defecto.

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.NumeroOrden)
    .HasColumnType("varchar(50)");

modelBuilder.Entity<PlanchaAcero>()
    .Property(p => p.Peso)
    .HasColumnType("decimal(18,3)");
```

CSHARP

La primera configuración cambia el tipo de NumeroOrden a varchar(50) en lugar de nvarchar(50). La segunda cambia el tipo de Peso a decimal(18,3). El uso de varchar en lugar de nvarchar reduce el espacio de almacenamiento pero no soporta caracteres Unicode.

## Configurar propiedades de solo lectura

Una propiedad de solo lectura se puede mapear a una columna con el método HasField o con el atributo [BackingField]. Es útil cuando se quiere encapsular el acceso a la propiedad.

```
private string _numeroOrden = string.Empty;

public string NumeroOrden
{
    get => _numeroOrden;
    private set => _numeroOrden = value;
}
```

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.NumeroOrden)
    .HasField("_numeroOrden")
    .UsePropertyAccessMode(PropertyAccessMode.Field);
```

CSHARP

La primera forma declara la propiedad con un campo de respaldo privado. La segunda configura EF Core para que use el campo en lugar de la propiedad.

## Configurar comentarios en columnas

EF Core permite añadir comentarios a las columnas con el método HasComment. Los comentarios se incluyen en el esquema de la base de datos y son útiles para documentar el propósito de cada columna.

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.NumeroOrden)
    .HasComment("Número único de la orden de fabricación en formato OF-YYYY-NNNN");
```

CSHARP

La primera configuración añade un comentario a la columna NumeroOrden. El comentario se incluye en el script de creación de la tabla.

## Configurar la collation de una columna

La collation determina cómo se comparan y ordenan las cadenas de texto. Se configura con el método UseCollation. Es útil cuando se quiere una collation específica para una columna.

```
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.Cliente)
    .UseCollation("SQL_Latin1_General_CP1_CI_AS");
```

CSHARP

La primera configuración establece la collation SQL\_Latin1\_General\_CP1\_CI\_AS para la columna Cliente. Esta collation es insensible a mayúsculas y minúsculas.

### Data Annotations vs Fluent API

Las Data Annotations son más sencillas de leer porque están junto a la propiedad. Sin embargo, tienen limitaciones: no permiten configurar relaciones complejas, no permiten configurar índices compuestos y no permiten configurar el comportamiento de eliminación. La Fluent API es más potente y permite configurar todo el modelo desde un solo lugar.

La recomendación general es usar Data Annotations para configuraciones simples y Fluent API para configuraciones complejas. En proyectos grandes, se suele usar Fluent API de forma exclusiva para mantener toda la configuración en un solo lugar.

CSHARP

```
// Data Annotations: simple y local
[Required]
[MaxLength(50)]
public string NumeroOrden { get; set; } = string.Empty;

// Fluent API: completo y centralizado
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.Property(o => o.NumeroOrden)
            .IsRequired()
            .HasMaxLength(50)
            .HasComment("Número único de la orden");
    });
}
```

La primera forma aplica los atributos sobre la propiedad. La segunda configura la propiedad desde OnModelCreating. Ambas producen el mismo resultado.

El método OnModelCreating

El método OnModelCreating es el lugar donde se configura el modelo con Fluent API. Se sobrescribe en el DbContext y recibe un modelBuilder que permite configurar entidades, propiedades, relaciones, índices y restricciones.

CSHARP

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);

        entity.Property(o => o.NumeroOrden)
            .IsRequired()
            .HasMaxLength(50);

        entity.Property(o => o.Cliente)
            .IsRequired()
            .HasMaxLength(200);

        entity.Property(o => o.FechaCreacion)
            .HasDefaultValueSql("GETDATE()");
    });

    modelBuilder.Entity<PlanchaAcero>(entity =>
    {
        entity.ToTable("PlanchasAcero");
        entity.HasKey(p => p.Id);

        entity.Property(p => p.Espesor)
            .HasPrecision(18, 2);

        entity.Property(p => p.Peso)
            .HasPrecision(18, 3);
    });
}
```

La primera línea declara el método. La segunda configura la entidad OrdenFabricacion. La tercera cambia el nombre de la tabla. La cuarta declara la clave primaria. La quinta configura la propiedad NumeroOrden. La sexta configura la propiedad Cliente. La séptima configura la propiedad FechaCreacion. La octava configura la entidad PlanchaAcero. La novena cambia el nombre de la tabla. La décima declara la clave primaria. La undécima configura la propiedad Espesor. La duodécima configura la propiedad Peso.

## El proyecto AceriaData

En el proyecto AceriaData, las entidades OrdenFabricacion, PlanchaAcero, Aleacion y EstadoOrden se configuran explícitamente en este punto. Se definen longitudes máximas para las propiedades de tipo string, se configuran propiedades requeridas y opcionales, se establece la precisión decimal para las propiedades de tipo decimal y se configuran valores por defecto. La configuración se realiza con Fluent API en el método OnModelCreating para mantener todo centralizado.

## Resumen de la teoría

Las propiedades se configuran con Data Annotations o con Fluent API.

Las propiedades requeridas se mapean a columnas NOT NULL.

Las propiedades opcionales se mapean a columnas NULL.

La longitud máxima de una propiedad string se configura con HasMaxLength.

La precisión decimal se configura con HasPrecision.

Los valores por defecto se configuran con HasDefaultValue o HasDefaultValueSql.

El tipo de columna se configura con HasColumnType.

Los comentarios de columna se configuran con HasComment.

La collation se configura con UseCollation.

La Fluent API se escribe en el método OnModelCreating.

La Fluent API tiene prioridad sobre las Data Annotations.

## Punto 2.3 - Relaciones uno a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configura explícitamente la relación uno a muchos entre OrdenFabricacion y PlanchaAcero en el proyecto AceriaData, definiendo la clave foránea, el comportamiento de eliminación y las propiedades de navegación.

### Objetivos de aprendizaje

Comprender qué es una relación uno a muchos y cómo se representa en el modelo.

Identificar los extremos principal y dependiente de la relación.

Configurar la relación con Fluent API: HasOne, WithMany, HasForeignKey y OnDelete.

Entender los distintos comportamientos de eliminación: Cascade, Restrict, SetNull y NoAction.

Configurar la relación con Data Annotations: [ForeignKey] y [InverseProperty].

Aplicar la configuración al proyecto AceriaData.

### Teoría

#### Qué es una relación uno a muchos

Una relación uno a muchos es la relación más común en los modelos relacionales. Una entidad principal se relaciona con muchas entidades dependientes, y cada entidad dependiente se relaciona con una sola entidad principal. En el dominio de la acería, una orden de fabricación puede tener muchas planchas de acero, y cada plancha pertenece a una sola orden.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public double Espesor { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera entidad tiene una colección Planchas. La segunda entidad tiene una propiedad de navegación Orden y una clave foránea OrdenId. EF Core detecta la relación por convención, pero en este punto se configura explícitamente para controlar su comportamiento.

## Los extremos de la relación

Una relación uno a muchos tiene dos extremos. El extremo principal es la entidad que contiene la colección. El extremo dependiente es la entidad que contiene la clave foránea. En el ejemplo, OrdenFabricacion es el extremo principal y PlanchaAcero es el extremo dependiente.

```
public class OrdenFabricacion
{
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
```

CSHARP

OrdenFabricacion contiene la colección Planchas, que representa el extremo "muchos". PlanchaAcero contiene la referencia Orden y la clave foránea OrdenId, que representan el extremo "uno". La clave foránea siempre reside en el extremo dependiente.

## La clave foránea

La clave foránea es la propiedad que almacena el valor de la clave primaria de la entidad principal. En el ejemplo, OrdenId almacena el valor de OrdenFabricacion.Id. EF Core detecta la clave foránea por convención cuando el nombre sigue el patrón <Navegacion>Id o <Entidad>Id.

```
public class PlanchaAcero
{
    public int OrdenId { get; set; }
```

CSHARP

La propiedad OrdenId se mapea a una columna OrdenId en la tabla PlanchasAcero. Esta columna es la que establece la relación con la tabla OrdenesFabricacion. Si la clave foránea no sigue el patrón por convención, se puede configurar explícitamente con HasForeignKey.

```
modelBuilder.Entity<PlanchaAcero>()
    .HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId);
```

CSHARP

La primera línea selecciona la entidad dependiente. La segunda indica que cada plancha tiene una orden. La tercera indica que cada orden tiene muchas planchas. La cuarta especifica la clave foránea.

## La propiedad de navegación

La propiedad de navegación es la propiedad que representa la relación desde el punto de vista del objeto. En el extremo principal, la propiedad de navegación es una colección. En el extremo dependiente, es una referencia.

```
public class OrdenFabricacion
{
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

public class PlanchaAcero
{
    public OrdenFabricacion Orden { get; set; } = null!;
```

CSHARP

Planchas es la propiedad de navegación de colección. Orden es la propiedad de navegación de referencia. EF Core usa estas propiedades para cargar las entidades relacionadas cuando se usa Include o cuando se accede a ellas con Lazy Loading.

## Configurar la relación con Fluent API

La relación uno a muchos se configura con la combinación de HasOne, WithMany y HasForeignKey. El orden de las llamadas puede variar, pero la semántica es la misma.

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(p => p.Orden)
        .WithMany(o => o.Planchas)
```

CSHARP

```
.HasForeignKey(p => p.OrdenId)
.OnDelete(DeleteBehavior.Cascade);
});
```

La primera línea selecciona la entidad `PlanchaAcero`. La segunda indica que cada plancha tiene una orden. La tercera indica que cada orden tiene muchas planchas. La cuarta especifica la clave foránea. La quinta configura el comportamiento de eliminación.

La configuración se puede escribir también desde el extremo principal:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasMany(o => o.Planchas)
        .WithOne(p => p.Orden)
        .HasForeignKey(p => p.OrdenId)
        .OnDelete(DeleteBehavior.Cascade);
});
```

CSHARP

La primera línea selecciona la entidad `OrdenFabricacion`. La segunda indica que cada orden tiene muchas planchas. La tercera indica que cada plancha tiene una orden. La cuarta especifica la clave foránea. La quinta configura el comportamiento de eliminación. Ambas formas producen el mismo resultado.

### Comportamientos de eliminación

El comportamiento de eliminación determina qué ocurre con las entidades dependientes cuando se elimina la entidad principal. EF Core soporta cuatro comportamientos: `Cascade`, `Restrict`, `SetNull` y `NoAction`.

`Cascade` elimina las entidades dependientes cuando se elimina la principal. Es el comportamiento por defecto para relaciones requeridas.

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .OnDelete(DeleteBehavior.Cascade);
```

CSHARP

La primera línea configura la relación. La segunda configura la eliminación en cascada. Al eliminar una orden, se eliminan todas sus planchas.

`Restrict` impide eliminar la entidad principal si existen entidades dependientes. La operación falla con una excepción.

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .OnDelete(DeleteBehavior.Restrict);
```

CSHARP

La primera línea configura la relación. La segunda configura la restricción. Al intentar eliminar una orden con planchas, la operación falla.

`SetNull` establece la clave foránea a null en las entidades dependientes cuando se elimina la principal. Solo se puede usar si la clave foránea es anulable.

```
public class PlanchaAcero
{
    public int? OrdenId { get; set; }
    public OrdenFabricacion? Orden { get; set; }
}
```

CSHARP

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .OnDelete(DeleteBehavior.SetNull);
```

CSHARP

La primera línea declara la clave foránea como anulable. La segunda declara la propiedad de navegación como anulable. La tercera configura la relación. La cuarta configura `SetNull`. Al eliminar una orden, las planchas quedan con `OrdenId` a null.

`NoAction` no hace nada en la base de datos. Es el comportamiento por defecto en algunos motores y deja la integridad referencial al código de la aplicación.

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .OnDelete(DeleteBehavior.NoAction);
```

CSHARP

La primera línea configura la relación. La segunda configura NoAction. Al eliminar una orden, las planchas no se modifican, lo que puede provocar una violación de integridad referencial si la clave foránea no es anulable.

#### Comportamiento por defecto

El comportamiento por defecto depende de la nulabilidad de la clave foránea. Si la clave foránea es no anulable (relación requerida), el comportamiento por defecto es Cascade. Si la clave foránea es anulable (relación opcional), el comportamiento por defecto es ClientSetNull, que es similar a SetNull pero se aplica en el cliente.

```
public class PlanchaAcero
{
    public int OrdenId { get; set; }
}
```

CSHARP

La clave foránea es no anulable, por lo que el comportamiento por defecto es Cascade. Al eliminar una orden, se eliminan sus planchas.

```
public class PlanchaAcero
{
    public int? OrdenId { get; set; }
}
```

CSHARP

La clave foránea es anulable, por lo que el comportamiento por defecto es ClientSetNull. Si las planchas dependientes están cargadas y seguidas por el mismo DbContext, EF Core establece OrdenId a null antes de guardar. Si los dependientes no están cargados, la restricción de base de datos no realiza esa propagación y el DELETE del principal puede fallar por integridad referencial; para propagación en la base de datos debe configurarse SetNull.

## Configurar la relación con Data Annotations

La relación uno a muchos se puede configurar con Data Annotations usando [ForeignKey] y [InverseProperty]. El atributo [ForeignKey] se aplica sobre la propiedad de navegación o sobre la clave foránea. El atributo [InverseProperty] se aplica sobre las propiedades de navegación cuando hay ambigüedad.

```
public class PlanchaAcero
{
    public int Id { get; set; }

    [ForeignKey(nameof(Orden))]
    public int OrdenId { get; set; }

    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea declara la clave primaria. La segunda aplica [ForeignKey] sobre OrdenId, indicando que es la clave foránea de la propiedad Orden. La tercera declara la propiedad de navegación.

El atributo [InverseProperty] se usa cuando hay varias propiedades de navegación entre las mismas entidades.

```
public class OrdenFabricacion
{
    public int Id { get; set; }

    [InverseProperty(nameof(PlanchaAcero.Orden))]
    public List<PlanchaAcero> Planchas { get; set; } = new();

    [InverseProperty(nameof(PlanchaAcero.OrdenSecundaria))]
    public List<PlanchaAcero> PlanchasSecundarias { get; set; } = new();
}
```

CSHARP

La primera propiedad de navegación se relaciona con PlanchaAcero.Orden. La segunda se relaciona con PlanchaAcero.OrdenSecundaria. Sin [InverseProperty], EF Core no sabría qué colección corresponde a qué referencia.

## Relaciones requeridas y opcionales

Una relación es requerida cuando la clave foránea es no anulable. Una relación es opcional cuando la clave foránea es anulable. La diferencia afecta al comportamiento de eliminación y a la validación de datos.

```
public class PlanchaAcero
{
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La clave foránea OrdenId es no anulable, por lo que la relación es requerida. Cada plancha debe tener una orden.

```
public class PlanchaAcero
{
    public int? OrdenId { get; set; }
    public OrdenFabricacion? Orden { get; set; }
}
```

CSHARP

La clave foránea OrdenId es anulable, por lo que la relación es opcional. Una plancha puede no tener orden.

## Configurar la relación como requerida

Para configurar explícitamente una relación como requerida, se usa el método `IsRequired` sobre la propiedad de navegación.

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .IsRequired();
```

CSHARP

La primera línea configura la relación. La segunda la marca como requerida. La columna OrdenId se crea como NOT NULL.

## Configurar la relación como opcional

Para configurar explícitamente una relación como opcional, se usa el método `IsRequired(false)`.

```
entity.HasOne(p => p.Orden)
    .WithMany(o => o.Planchas)
    .HasForeignKey(p => p.OrdenId)
    .IsRequired(false);
```

CSHARP

La primera línea configura la relación. La segunda la marca como opcional. La columna OrdenId se crea como NULL.

Cargar entidades relacionadas

Las entidades relacionadas se cargan con `Include` y `ThenInclude`. `Include` carga la colección o la referencia.

`ThenInclude` carga las propiedades de navegación de las entidades incluidas.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .FirstOrDefault(o => o.Id == 1);
```

CSHARP

La primera línea selecciona la entidad principal. La segunda incluye la colección Planchas. La tercera filtra por Id y devuelve la primera coincidencia. Sin `Include`, la colección Planchas estaría vacía.

```
var plancha = context.PlanchasAcero
    .Include(p => p.Orden)
    .FirstOrDefault(p => p.Id == 1);
```

CSHARP

La primera línea selecciona la entidad dependiente. La segunda incluye la referencia Orden. La tercera filtra por Id y devuelve la primera coincidencia. Sin `Include`, la propiedad Orden estaría a null.

## El proyecto AceriaData

En el proyecto AceriaData, la relación uno a muchos entre OrdenFabricacion y PlanchaAcero se configura explícitamente en este punto. Se define la clave foránea OrdenId, se configura el comportamiento de eliminación como Cascade y se establece la relación como requerida. La configuración se realiza con Fluent API en el método OnModelCreating.

## Resumen de la teoría

Una relación uno a muchos tiene un extremo principal y un extremo dependiente.

La clave foránea reside en el extremo dependiente.

La propiedad de navegación de colección está en el extremo principal.

La propiedad de navegación de referencia está en el extremo dependiente.

La relación se configura con HasOne, WithMany y HasForeignKey.

El comportamiento de eliminación se configura con OnDelete.

Los comportamientos son Cascade, Restrict, SetNull y NoAction.

El comportamiento por defecto es Cascade para relaciones requeridas y ClientSetNull para opcionales.

Las relaciones se pueden configurar con Data Annotations usando [ForeignKey] y [InverseProperty].

Las entidades relacionadas se cargan con Include y ThenInclude.

## Punto 2.4 - Relaciones uno a uno

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configura explícitamente la relación uno a uno entre OrdenFabricacion y DetalleOrden en el proyecto AceriaData, definiendo el índice único sobre la clave foránea, el comportamiento de eliminación y las propiedades de navegación.

## Objetivos de aprendizaje

Comprender qué es una relación uno a uno y cómo se diferencia de la relación uno a muchos.

Identificar los extremos principal y dependiente de una relación uno a uno.

Comprender el papel del índice único sobre la clave foránea.

Configurar la relación con Fluent API: HasOne, WithOne y HasForeignKey.

Configurar la relación con clave compartida y con clave foránea independiente.

Configurar el comportamiento de eliminación en una relación uno a uno.

Cargar entidades relacionadas uno a uno con Include.

Aplicar la configuración al proyecto AceriaData.

## Teoría

### Qué es una relación uno a uno

Una relación uno a uno es aquella en la que una entidad principal se relaciona con una sola entidad dependiente, y cada entidad dependiente se relaciona con una sola entidad principal. En el dominio de la acería, una orden de fabricación puede tener un solo detalle de orden, y cada detalle de orden pertenece a una sola orden. El detalle contiene información que complementa a la orden, como la composición química del acero, la temperatura de la colada o las notas de producción.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
}
```

CSHARP

```

    public DetalleOrden? Detalle { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

```

La primera entidad tiene una propiedad de navegación de referencia Detalle. La segunda entidad tiene una propiedad de navegación de referencia Orden y una clave foránea OrdenId. La diferencia con la relación uno a muchos es que aquí ambas propiedades de navegación son de referencia, no hay colecciones.

Diferencia con la relación uno a muchos

En una relación uno a muchos, el extremo principal tiene una colección y el extremo dependiente tiene una referencia. En una relación uno a uno, ambos extremos tienen una referencia. Esa es la diferencia esencial.

```

// Uno a muchos
public class OrdenFabricacion
{
    public List<PlanchaAcero> Planchas { get; set; } = new();
}

// Uno a uno
public class OrdenFabricacion
{
    public DetalleOrden? Detalle { get; set; }
}

```

CSHARP

La primera forma declara una colección de planchas. La segunda forma declara una referencia a un detalle. EF Core interpreta la colección como uno a muchos y la referencia como uno a uno.

Los extremos de la relación uno a uno

Una relación uno a uno tiene un extremo principal y un extremo dependiente. El extremo principal es la entidad que no contiene la clave foránea. El extremo dependiente es la entidad que contiene la clave foránea. En el ejemplo, OrdenFabricacion es el extremo principal y DetalleOrden es el extremo dependiente porque contiene OrdenId.

```

public class OrdenFabricacion
{
    public int Id { get; set; }
    public DetalleOrden? Detalle { get; set; }
}

public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}

```

CSHARP

OrdenFabricacion no tiene clave foránea hacia DetalleOrden. DetalleOrden sí tiene clave foránea hacia OrdenFabricacion. Por eso DetalleOrden es el extremo dependiente.

El índice único sobre la clave foránea

Lo que convierte una relación uno a muchos en una relación uno a uno es el índice único sobre la clave foránea. Sin el índice único, la clave foránea puede repetirse y la relación se comporta como uno a muchos. Con el índice único, cada valor de la clave foránea aparece una sola vez y la relación es uno a uno.

```
CREATE UNIQUE INDEX IX_DetallesOrden_OrdenId ON DetallesOrden(OrdenId);
```

SQL

La primera línea crea un índice único sobre la columna OrdenId. Este índice garantiza que no puede haber dos detalles con el mismo OrdenId. Si se intenta insertar un segundo detalle para la misma orden, la base de datos lanza una excepción de violación de unicidad.

EF Core genera este índice automáticamente cuando se configura la relación con `HasForeignKey<TDependiente>` y `WithOne`. El genérico `<TDependiente>` indica cuál es el extremo dependiente, y EF Core crea el índice único sobre la clave foránea.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId);
```

CSHARP

La primera línea indica que cada detalle tiene una orden. La segunda indica que cada orden tiene un detalle. La tercera especifica la clave foránea usando el genérico <DetalleOrden>, que marca DetalleOrden como el extremo dependiente. EF Core crea el índice único sobre OrdenId.

#### Patrón de clave compartida

Existe un segundo patrón para configurar relaciones uno a uno: el patrón de clave compartida. En este patrón, la clave primaria de la entidad dependiente también es la clave foránea hacia la entidad principal. Este patrón es útil cuando la entidad dependiente no tiene sentido sin la principal y se quiere que compartan el mismo identificador.

```
public class DetalleOrden
{
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

En este ejemplo, DetalleOrden no tiene una propiedad Id propia. Su clave primaria es OrdenId, que también es la clave foránea hacia OrdenFabricacion. EF Core detecta este patrón por convención cuando la propiedad que actúa como clave foránea también es la única candidata a clave primaria.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId);
```

CSHARP

La primera línea configura la relación. La segunda indica que es uno a uno. La tercera especifica la clave foránea, que también es la clave primaria. EF Core no necesita crear un índice único adicional porque la clave primaria ya es única.

El patrón de clave compartida tiene ventajas y desventajas. La ventaja es que no se necesita una columna adicional para la clave primaria y la relación es más compacta. La desventaja es que la entidad dependiente no puede existir sin la principal y no se puede cambiar la relación sin cambiar la clave primaria.

#### Patrón de clave foránea independiente

El patrón más común es el de clave foránea independiente. En este patrón, la entidad dependiente tiene su propia clave primaria y una clave foránea separada hacia la entidad principal.

```
public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public string ComposicionQuimica { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea declara Id como clave primaria. La segunda declara OrdenId como clave foránea. La tercera declara la propiedad de navegación. EF Core crea un índice único sobre OrdenId para garantizar la unicidad.

Este patrón es más flexible porque la entidad dependiente conserva una clave primaria propia y la relación puede cambiar sin modificar esa identidad. Si la clave foránea es requerida, el dependiente sigue necesitando una entidad principal válida para poder persistirse. Es el patrón que se usa en el proyecto AceriaData.

## Configurar la relación con Fluent API

La relación uno a uno se configura con la combinación de HasOne, WithOne yHasForeignKey<TDependiente>. El orden de las llamadas puede variar, pero la semántica es la misma.

```
modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.HasOne(d => d.Orden)
        .WithOne(o => o.Detalle)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

CSHARP

La primera línea selecciona la entidad `DetalleOrden`. La segunda indica que cada detalle tiene una orden. La tercera indica que cada orden tiene un detalle. La cuarta especifica la clave foránea usando el genérico. La quinta configura el comportamiento de eliminación. La sexta marca la relación como requerida.

La configuración también se puede escribir desde el extremo principal:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasOne(o => o.Detalle)
        .WithOne(d => d.Orden)
        .HasForeignKey<DetalleOrden>(d => d.OrdenId)
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();
});
```

CSHARP

La primera línea selecciona la entidad `OrdenFabricacion`. La segunda indica que cada orden tiene un detalle. La tercera indica que cada detalle tiene una orden. La cuarta especifica la clave foránea. La quinta configura el comportamiento de eliminación. La sexta marca la relación como requerida. Ambas formas producen el mismo resultado.

#### La importancia del genérico en `HasForeignKey`

El método `HasForeignKey` tiene dos sobrecargas. La primera acepta una expresión lambda que indica la propiedad de la clave foránea. La segunda acepta un genérico que indica el tipo de la entidad dependiente. En relaciones uno a uno, es importante usar la sobrecarga con genérico para que EF Core sepa cuál es el extremo dependiente.

```
// Sin genérico: EF Core no sabe cuál es el dependiente
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey(d => d.OrdenId);

// Con genérico: EF Core sabe que DetalleOrden es el dependiente
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId);
```

CSHARP

La primera forma puede provocar que EF Core interprete la relación de forma incorrecta si no puede deducir el extremo dependiente. La segunda forma elimina la ambigüedad al indicar explícitamente que `DetalleOrden` es el extremo dependiente.

#### Comportamientos de eliminación en uno a uno

Los comportamientos de eliminación en una relación uno a uno son los mismos que en una relación uno a muchos: `Cascade`, `Restrict`, `SetNull` y `NoAction`. El comportamiento por defecto depende de la nulabilidad de la clave foránea.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .OnDelete(DeleteBehavior.Cascade);
```

CSHARP

La primera línea configura la relación. La segunda indica que es uno a uno. La tercera especifica la clave foránea. La cuarta configura la eliminación en cascada. Al eliminar una orden, se elimina también su detalle.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .OnDelete(DeleteBehavior.Restrict);
```

CSHARP

La primera línea configura la relación. La segunda indica que es uno a uno. La tercera especifica la clave foránea. La cuarta configura la restricción. Al intentar eliminar una orden con detalle, la operación falla.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .OnDelete(DeleteBehavior.SetNull);
```

CSHARP

La primera línea configura la relación. La segunda indica que es uno a uno. La tercera especifica la clave foránea. La cuarta configura `SetNull`. Este comportamiento solo es válido si la clave foránea es anulable.

### Relación uno a uno requerida y opcional

Una relación uno a uno es requerida cuando la clave foránea es no anulable. Una relación uno a uno es opcional cuando la clave foránea es anulable.

```
public class DetalleOrden
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La clave foránea OrdenId es no anulable, por lo que la relación es requerida. Cada detalle debe tener una orden. La orden puede o no tener detalle, dependiendo de la nulabilidad de la propiedad de navegación en el extremo principal.

```
public class DetalleOrden
{
    public int Id { get; set; }
    public int? OrdenId { get; set; }
    public OrdenFabricacion? Orden { get; set; }
}
```

CSHARP

La clave foránea OrdenId es anulable, por lo que la relación es opcional. Un detalle puede no tener orden. Este patrón es menos común porque en una relación uno a uno el detalle suele depender de la orden.

### Configurar la relación como requerida

Para configurar explícitamente una relación uno a uno como requerida, se usa el método `IsRequired` sobre la propiedad de navegación del extremo dependiente.

```
entity.HasOne(d => d.Orden)
    .WithOne(o => o.Detalle)
    .HasForeignKey<DetalleOrden>(d => d.OrdenId)
    .IsRequired();
```

CSHARP

La primera línea configura la relación. La segunda indica que es uno a uno. La tercera especifica la clave foránea. La cuarta marca la relación como requerida. La columna OrdenId se crea como NOT NULL.

#### Opcionalidad desde el principal

Una OrdenFabricacion puede no tener todavía un DetalleOrden, por lo que su navegación puede declararse nullable:

```
public DetalleOrden? Detalle { get; set; }
```

CSHARP

Eso no hace nullable la clave foránea del dependiente. Si existe un DetalleOrden, su OrdenId es obligatorio y la relación se configura una única vez desde el dependiente con `HasForeignKey<DetalleOrden>(...).IsRequired()`.

### Configurar la relación con Data Annotations

La relación uno a uno se puede configurar con Data Annotations usando `[ForeignKey]` y `[InverseProperty]`.

```
public class DetalleOrden
{
    public int Id { get; set; }

    [ForeignKey(nameof(Orden))]
    public int OrdenId { get; set; }

    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea declara la clave primaria. La segunda aplica `[ForeignKey]` sobre OrdenId, indicando que es la clave foránea de la propiedad Orden. La tercera declara la propiedad de navegación.

El atributo `[InverseProperty]` se usa cuando hay varias propiedades de navegación entre las mismas entidades.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
```

CSHARP

```
[InverseProperty(nameof(DetalleOrden.Orden))]
public DetalleOrden? Detalle { get; set; }
}
```

La primera línea declara la clave primaria. La segunda aplica [InverseProperty] sobre Detalle, indicando que se relaciona con DetalleOrden.Orden. La tercera declara la propiedad de navegación.

Cargar entidades relacionadas uno a uno

Las entidades relacionadas uno a uno se cargan con Include. La diferencia con la relación uno a muchos es que la propiedad de navegación es de referencia, no de colección.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Detalle)
    .FirstOrDefault(o => o.NumeroOrden == "OF-2024-0001");
```

CSHARP

La primera línea selecciona la entidad principal. La segunda incluye la referencia Detalle. La tercera filtra por número de orden y devuelve la primera coincidencia. Sin Include, la propiedad Detalle estaría a null.

```
var detalle = context.DetallesOrden
    .Include(d => d.Orden)
    .FirstOrDefault(d => d.Id == 1);
```

CSHARP

La primera línea selecciona la entidad dependiente. La segunda incluye la referencia Orden. La tercera filtra por Id y devuelve la primera coincidencia. Sin Include, la propiedad Orden estaría a null.

Insertar entidades relacionadas uno a uno

Al insertar una entidad con su entidad relacionada, se puede usar la propiedad de navegación para que EF Core propague la clave foránea automáticamente.

```
var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-2024-0001",
    Cliente = "Constructora del Norte",
    Detalle = new DetalleOrden
    {
        ComposicionQuimica = "C: 0.45%, Mn: 0.75%",
        TemperaturaColada = 1550.5
    }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La primera línea crea la orden con su detalle anidado. La segunda línea registra la orden y, por propagación, el detalle. La tercera línea inserta ambas entidades y propaga la clave foránea.

También se pueden insertar por separado, guardando primero la orden y después el detalle.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte" };
context.OrdenesFabricacion.Add(orden);
context.SaveChanges();

var detalle = new DetalleOrden { OrdenId = orden.Id, ComposicionQuimica = "C: 0.45%", TemperaturaColada = 1550.5 };
context.DetallesOrden.Add(detalle);
context.SaveChanges();
```

CSHARP

La primera línea crea la orden. La segunda la registra. La tercera la inserta y obtiene el Id. La cuarta crea el detalle con la clave foránea. La quinta lo registra. La sexta lo inserta. Ambas formas producen el mismo resultado.

Eliminar entidades relacionadas uno a uno

Al eliminar una entidad principal, el comportamiento de eliminación determina qué ocurre con la entidad dependiente. Con Cascade, el detalle se elimina automáticamente. Con Restrict, la operación falla si existe detalle.

```
var orden = context.OrdenesFabricacion.Include(o => o.Detalle).FirstOrDefault(o => o.Id == 1);
context.OrdenesFabricacion.Remove(orden!);
context.SaveChanges();
```

CSHARP

La primera línea carga la orden con su detalle. La segunda la marca para eliminar. La tercera ejecuta el DELETE. Si el comportamiento es Cascade, EF Core genera también el DELETE del detalle.

### Restricción de unicidad en la base de datos

Aunque EF Core genera el índice único, es importante que la base de datos también lo aplique. El índice único es la garantía de que la relación es uno a uno y no uno a muchos. Sin el índice, dos detalles podrían apuntar a la misma orden.

```
CREATE UNIQUE INDEX IX_DetallesOrden_OrdenId ON DetallesOrden(OrdenId);
```

SQL

La primera línea crea el índice único. Si se intenta insertar un segundo detalle para la misma orden, la base de datos rechaza la operación con un error de violación de unicidad.

## El proyecto AceriaData

En el proyecto AceriaData, la relación uno a uno entre OrdenFabricacion y DetalleOrden se configura explícitamente en este punto. La entidad DetalleOrden ya existe desde el punto 2.3, pero su relación con OrdenFabricacion se configura ahora como uno a uno con índice único sobre OrdenId. Se define el comportamiento de eliminación como Cascade y se marca la relación como requerida. La configuración se realiza con Fluent API en el método OnModelCreating.

## Resumen de la teoría

Una relación uno a uno tiene una referencia en cada extremo, no colecciones.

El extremo dependiente contiene la clave foránea.

Lo que convierte la relación en uno a uno es el índice único sobre la clave foránea.

El patrón de clave compartida usa la clave primaria como clave foránea.

El patrón de clave foránea independiente usa una clave primaria propia y una clave foránea separada.

La relación se configura con HasOne, WithOne y HasForeignKey<TDependiente>.

El genérico en HasForeignKey indica cuál es el extremo dependiente.

El comportamiento de eliminación se configura con OnDelete.

La relación puede ser requerida o opcional según la nulabilidad de la clave foránea.

Las entidades relacionadas se cargan con Include.

Las entidades relacionadas se insertan con la propiedad de navegación o con la clave foránea.

El índice único garantiza la unicidad en la base de datos.

## Punto 2.5 - Relaciones muchos a muchos

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configura explícitamente la relación muchos a muchos entre OrdenFabricacion y Aleacion en el proyecto AceriaData, tanto con la tabla intermedia implícita como con una entidad intermedia explícita que contiene datos adicionales.

## Objetivos de aprendizaje

Comprender qué es una relación muchos a muchos y cómo se representa en el modelo relacional.

Diferenciar entre tabla intermedia implícita y entidad intermedia explícita.

Configurar la relación muchos a muchos por convención.

Configurar la relación muchos a muchos con Fluent API.

Configurar una entidad intermedia explícita con propiedades adicionales.

Utilizar las skip navigations para consultar la relación directamente.

Insertar y consultar entidades relacionadas muchos a muchos.

Aplicar la configuración al proyecto AceriaData.

## Teoría

### Qué es una relación muchos a muchos

Una relación muchos a muchos es aquella en la que una entidad principal se relaciona con muchas entidades de otro tipo, y cada entidad de ese otro tipo se relaciona con muchas entidades del primero. En el dominio de la acería, una orden de fabricación puede utilizar varias aleaciones de acero, y una aleación puede utilizarse en varias órdenes de fabricación. No hay una relación de dependencia directa entre ellas: ambas existen de forma independiente y se relacionan a través de una tabla intermedia.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public List<Aleacion> Aleaciones { get; set; } = new();
}

public class Aleacion
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public List<OrdenFabricacion> Ordenes { get; set; } = new();
}
```

CSHARP

La primera entidad tiene una colección Aleaciones. La segunda entidad tiene una colección Ordenes. Ambas propiedades de navegación son de colección. Esta es la característica que distingue la relación muchos a muchos de la relación uno a muchos, donde solo un extremo tiene colección.

El modelo relacional subyacente

En una base de datos relacional, una relación muchos a muchos no se puede representar directamente entre dos tablas. Se necesita una tercera tabla, llamada tabla intermedia o tabla de unión, que contenga las claves foráneas de ambas tablas. Esta tabla intermedia tiene una clave primaria compuesta formada por las dos claves foráneas.

```
CREATE TABLE OrdenesFabricacion (
    Id INT PRIMARY KEY IDENTITY(1,1),
    NumeroOrden NVARCHAR(50) NOT NULL
);

CREATE TABLE Aleaciones (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Nombre NVARCHAR(100) NOT NULL
);

CREATE TABLE AleacionOrdenFabricacion (
    AleacionesId INT NOT NULL,
    OrdenesId INT NOT NULL,
    PRIMARY KEY (AleacionesId, OrdenesId),
    FOREIGN KEY (AleacionesId) REFERENCES Aleaciones(Id),
    FOREIGN KEY (OrdenesId) REFERENCES OrdenesFabricacion(Id)
);
```

SQL

La primera tabla almacena las órdenes. La segunda almacena las aleaciones. La tercera tabla, AleacionOrdenFabricacion, es la tabla intermedia. Contiene dos columnas: AleacionesId y OrdenesId. La clave primaria es compuesta, formada por ambas columnas. Cada fila representa una relación entre una orden y una aleación.

Tabla intermedia implícita

EF Core puede crear la tabla intermedia automáticamente si no se necesita almacenar información adicional en ella. En este caso, la tabla intermedia solo contiene las dos claves foráneas y no tiene entidad propia en el modelo. Se conoce como tabla intermedia implícita o join table.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public List<Aleacion> Aleaciones { get; set; } = new();
}

public class Aleacion
{
    public int Id { get; set; }
    public List<OrdenFabricacion> Ordenes { get; set; } = new();
}
```

CSHARP

Con este código, sin ninguna configuración adicional, EF Core detecta la relación muchos a muchos y crea una tabla intermedia llamada `AleacionOrdenFabricacion` con las columnas `AleacionesId` y `OrdenesId`. El nombre de la tabla se forma concatenando los nombres de las dos entidades en orden alfabético. Las columnas se forman concatenando el nombre de la entidad con `Id`.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasMany(o => o.Aleaciones)
    .WithMany(a => a.Ordenes)
    .UsingEntity(j => j.ToTable("OrdenesAleaciones"));
```

CSHARP

La primera línea selecciona la entidad `OrdenFabricacion`. La segunda indica que una orden tiene muchas aleaciones. La tercera indica que una aleación tiene muchas órdenes. La cuarta configura la tabla intermedia con el nombre `OrdenesAleaciones`. El método `UsingEntity` permite configurar la tabla intermedia sin necesidad de crear una entidad explícita.

#### Tabla intermedia explícita

Cuando se necesita almacenar información adicional en la tabla intermedia, como la fecha de asignación de la aleación a la orden o la cantidad utilizada, se crea una entidad intermedia explícita. Esta entidad representa la tabla intermedia y contiene las dos claves foráneas más las propiedades adicionales.

```
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; }
    public decimal CantidadUtilizada { get; set; }
    public OrdenFabricacion Orden { get; set; } = null!;
    public Aleacion Aleacion { get; set; } = null!;
}
```

CSHARP

La primera línea declara la entidad intermedia. La segunda declara la clave foránea hacia `OrdenFabricacion`. La tercera declara la clave foránea hacia `Aleacion`. La cuarta declara la fecha de asignación. La quinta declara la cantidad utilizada. La sexta y la séptima declaran las propiedades de navegación hacia las entidades principales.

Con la entidad intermedia explícita, las entidades principales ya no tienen colecciones de la otra entidad directamente. En su lugar, tienen colecciones de la entidad intermedia.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}

public class Aleacion
{
    public int Id { get; set; }
    public List<OrdenAleacion> OrdenesAleaciones { get; set; } = new();
}
```

CSHARP

La primera entidad tiene una colección de `OrdenAleacion`. La segunda entidad también tiene una colección de `OrdenAleacion`. La entidad intermedia es el punto central de la relación. Para acceder a las aleaciones de una orden, se navega a través de `OrdenesAleaciones` y luego a `Aleacion`.

### Configurar la entidad intermedia explícita

La entidad intermedia explícita se configura con `HasOne`, `WithMany` y `HasForeignKey` para cada una de las dos relaciones. Además, se configura la clave primaria compuesta con `HasKey`.

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.ToTable("OrdenesAleaciones");
    entity.HasKey(oa => new { oa.OrdenFabricacionId, oa.AleacionId });

    entity.HasOne(oa => oa.Orden)
        .WithMany(o => o.OrdenesAleaciones)
        .HasForeignKey(oa => oa.OrdenFabricacionId)
        .OnDelete(DeleteBehavior.Cascade);

    entity.HasOne(oa => oa.Aleacion)
        .WithMany(a => a.OrdenesAleaciones)
        .HasForeignKey(oa => oa.AleacionId)
});
```

CSHARP

```
.onDelete(DeleteBehavior.Restrict);
});
```

La primera línea selecciona la entidad intermedia. La segunda establece el nombre de la tabla. La tercera declara la clave primaria compuesta. La cuarta configura la relación con OrdenFabricacion. La quinta indica que una orden tiene muchas entradas en la tabla intermedia. La sexta especifica la clave foránea. La séptima configura la eliminación en cascada. La octava configura la relación con Aleacion. La novena indica que una aleación tiene muchas entradas en la tabla intermedia. La décima especifica la clave foránea. La undécima configura la eliminación restrictiva.

#### Skip navigations

EF Core 5 y versiones posteriores introdujeron las skip navigations, que permiten navegar directamente entre las dos entidades principales de una relación muchos a muchos sin pasar por la entidad intermedia. Las skip navigations se configuran con UsingEntity y son útiles cuando se quiere consultar la relación de forma directa.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasMany(o => o.Aleaciones)
    .WithMany(a => a.Ordenes)
    .UsingEntity<OrdenAleacion>()
    {
        oa => oa.HasOne(x => x.Aleacion).WithMany().HasForeignKey(x => x.AleacionId),
        oa => oa.HasOne(x => x.Orden).WithMany().HasForeignKey(x => x.OrdenFabricacionId),
        oa =>
        {
            oa.ToTable("OrdenesAleaciones");
            oa.HasKey(x => new { x.OrdenFabricacionId, x.AleacionId });
        }
    };
```

CSHARP

La primera línea selecciona la entidad OrdenFabricacion. La segunda indica que una orden tiene muchas aleaciones. La tercera indica que una aleación tiene muchas órdenes. La cuarta configura la entidad intermedia con UsingEntity<OrdenAleacion>. La quinta configura la relación con Aleacion. La sexta configura la relación con OrdenFabricacion. La séptima configura la tabla y la clave primaria. Con esta configuración, se pueden usar ambas formas de navegación: a través de las colecciones de la entidad intermedia o directamente entre OrdenFabricacion y Aleacion.

### Configurar la tabla intermedia implícita con nombre personalizado

Si se usa la tabla intermedia implícita, se puede personalizar el nombre de la tabla y de las columnas con UsingEntity.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasMany(o => o.Aleaciones)
    .WithMany(a => a.Ordenes)
    .UsingEntity<Dictionary<string, object>>()
    {
        "OrdenesAleaciones",
        j => j.HasOne<Aleacion>().WithMany().HasForeignKey("AleacionId"),
        j => j.HasOne<OrdenFabricacion>().WithMany().HasForeignKey("OrdenFabricacionId"),
        j =>
        {
            j.ToTable("OrdenesAleaciones");
            j.HasKey("OrdenFabricacionId", "AleacionId");
        }
    };
```

CSHARP

La primera línea selecciona la entidad OrdenFabricacion. La segunda indica que una orden tiene muchas aleaciones. La tercera indica que una aleación tiene muchas órdenes. La cuarta usa un diccionario para representar la tabla intermedia. La quinta configura la relación con Aleacion. La sexta configura la relación con OrdenFabricacion. La séptima configura la tabla y la clave primaria.

#### Insertar entidades relacionadas muchos a muchos

Al insertar entidades relacionadas muchos a muchos, se pueden usar las propiedades de navegación para que EF Core inserte también las filas de la tabla intermedia.

```
var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045" };
var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140" };

var orden = new OrdenFabricacion
{
    NumeroOrden = "OF-2024-0001",
    Cliente = "Constructora del Norte",
    Aleaciones = new List<Aleacion> { aleacion1, aleacion2 }
};

context.OrdenesFabricacion.Add(orden);
context.SaveChanges();
```

CSHARP

La primera línea crea la primera aleación. La segunda crea la segunda aleación. La tercera crea la orden. La cuarta asigna las aleaciones a la orden. La quinta registra la orden y, por propagación, las aleaciones y las filas de la tabla intermedia. La sexta inserta todo en una sola transacción.

Con entidad intermedia explícita, la inserción se realiza a través de la entidad intermedia.

```
var orden = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del Norte" };
var aleacion = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045" };

var ordenAleacion = new OrdenAleacion
{
    Orden = orden,
    Aleacion = aleacion,
    FechaAsignacion = DateTime.Now,
    CantidadUtilizada = 1500.5m
};

context.OrdenesAleaciones.Add(ordenAleacion);
context.SaveChanges();
```

CSHARP

La primera línea crea la orden. La segunda crea la aleación. La tercera crea la entidad intermedia. La cuarta asigna la orden. La quinta asigna la aleación. La sexta asigna la fecha. La séptima asigna la cantidad. La octava registra la entidad intermedia. La novena inserta todo.

Consultar entidades relacionadas muchos a muchos

Con tabla intermedia implícita, las consultas se realizan con Include.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Aleaciones)
    .FirstOrDefault(o => o.NumeroOrden == "OF-2024-0001");

foreach (var aleacion in orden!.Aleaciones)
{
    Console.WriteLine($"Aleación: {aleacion.Nombre} ({aleacion.Codigo})");
}
```

CSHARP

La primera línea inicia la consulta. La segunda incluye la colección de aleaciones. La tercera filtra por número de orden. La cuarta itera sobre las aleaciones. La quinta muestra el nombre y el código de cada aleación.

Con entidad intermedia explícita, las consultas se realizan con Include y ThenInclude.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.OrdenesAleaciones)
    .ThenInclude(oa => oa.Aleacion)
    .FirstOrDefault(o => o.NumeroOrden == "OF-2024-0001");

foreach (var ordenAleacion in orden!.OrdenesAleaciones)
{
    Console.WriteLine($"Aleación: {ordenAleacion.Aleacion.Nombre} | Cantidad: {ordenAleacion.CantidadUtilizada}");
}
```

CSHARP

La primera línea inicia la consulta. La segunda incluye la colección de entidades intermedias. La tercera incluye la aleación de cada entidad intermedia. La cuarta filtra por número de orden. La quinta itera sobre las entidades intermedias. La sexta muestra el nombre de la aleación y la cantidad utilizada.

Eliminar entidades relacionadas muchos a muchos

Al eliminar una entidad principal, las filas de la tabla intermedia se eliminan según el comportamiento de eliminación configurado. Con Cascade, las filas se eliminan automáticamente. Con Restrict, la operación falla si existen filas en la tabla intermedia.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Aleaciones)
    .FirstOrDefault(o => o.NumeroOrden == "OF-2024-0001");

context.OrdenesFabricacion.Remove(orden!);
context.SaveChanges();
```

CSHARP

La primera línea inicia la consulta. La segunda incluye las aleaciones. La tercera filtra por número de orden. La cuarta marca la orden para eliminar. La quinta ejecuta el DELETE de la orden y de las filas de la tabla intermedia.

Actualizar entidades relacionadas muchos a muchos

Para actualizar una relación muchos a muchos, se añaden o se eliminan elementos de la colección de navegación.

CSHARP

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Aleaciones)
    .FirstOrDefault(o => o.NumeroOrden == "0F-2024-0001");

var nuevaAleacion = context.Aleaciones.FirstOrDefault(a => a.Codigo == "A4140");

orden!.Aleaciones.Add(nuevaAleacion!);
context.SaveChanges();
```

La primera línea inicia la consulta. La segunda incluye las aleaciones. La tercera filtra por número de orden. La cuarta busca la nueva aleación. La quinta añade la aleación a la colección. La sexta inserta la nueva fila en la tabla intermedia.

Para eliminar una relación, se quita el elemento de la colección.

CSHARP

```
var aleacion = orden.Aleaciones.FirstOrDefault(a => a.Codigo == "A4140");
orden.Aleaciones.Remove(aleacion!);
context.SaveChanges();
```

La primera línea busca la aleación en la colección. La segunda la elimina de la colección. La tercera elimina la fila de la tabla intermedia.

## El proyecto AceriaData

En el proyecto AceriaData, la relación muchos a muchos entre OrdenFabricacion y Aleacion se configura con una entidad intermedia explícita llamada OrdenAleacion. Esta entidad contiene la fecha de asignación y la cantidad utilizada, que son datos relevantes para el negocio. La configuración se realiza con Fluent API en el método OnModelCreating. Se configuran las dos relaciones uno a muchos desde la entidad intermedia hacia las entidades principales, y se define la clave primaria compuesta.

## Resumen de la teoría

Una relación muchos a muchos se representa con una tabla intermedia.

La tabla intermedia contiene las claves foráneas de ambas entidades.

La tabla intermedia puede ser implícita o explícita.

La tabla implícita se crea automáticamente por convención.

La tabla explícita se crea con una entidad intermedia que puede tener propiedades adicionales.

La tabla intermedia se configura con UsingEntity.

La entidad intermedia se configura con HasOne, WithMany, HasForeignKey y HasKey.

Las skip navigations permiten navegar directamente entre las entidades principales.

Las entidades relacionadas se insertan con las propiedades de navegación.

Las entidades relacionadas se consultan con Include y ThenInclude.

Las relaciones se actualizan añadiendo o eliminando elementos de la colección.

La eliminación en cascada elimina las filas de la tabla intermedia.

## Punto 2.6 - Configuración mediante Data Annotations

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configuran las entidades del proyecto AceriaData con Data Annotations, como alternativa a Fluent API, aplicando atributos para claves, longitudes, relaciones y validaciones directamente sobre las propiedades.

## Objetivos de aprendizaje

Comprender qué son las Data Annotations y cómo se aplican.

Conocer los atributos principales: [Key], [Required], [MaxLength], [StringLength], [Column], [Table], [ForeignKey], [InverseProperty], [NotMapped], [Index], [Precision], [DatabaseGenerated].

Diferenciar entre Data Annotations y Fluent API.

Comprender la prioridad de configuración: Fluent API > Data Annotations > Convenciones.

Aplicar Data Annotations a las entidades del proyecto AceriaData.

Coexistir Data Annotations con Fluent API en el mismo modelo.

## Teoría

### Qué son las Data Annotations

Las Data Annotations son atributos de .NET que se aplican directamente sobre las clases y propiedades de las entidades para configurar el modelo de EF Core. Son parte del espacio de nombres System.ComponentModel.DataAnnotations y System.ComponentModel.DataAnnotations.Schema. Su principal ventaja es que mantienen la configuración junto a la propiedad que configuran, lo que facilita la lectura y el mantenimiento en modelos sencillos.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    [MaxLength(200)]
    public string Cliente { get; set; } = string.Empty;
}
```

CSHARP

La primera línea importa el espacio de nombres System.ComponentModel.DataAnnotations, que contiene los atributos de validación y configuración. La segunda línea importa System.ComponentModel.DataAnnotations.Schema, que contiene los atributos de mapeo a la base de datos. El atributo [Table("OrdenesFabricacion")] sobre la clase establece el nombre de la tabla. El atributo [Key] sobre Id declara la clave primaria. El atributo [Required] sobre NumeroOrden marca la propiedad como requerida. El atributo [MaxLength(50)] establece la longitud máxima en 50 caracteres.

EF Core lee estos atributos al construir el modelo y los aplica como si se hubieran configurado con Fluent API. El resultado es el mismo: la tabla se llama OrdenesFabricacion, la columna Id es la clave primaria, la columna NumeroOrden es nvarchar(50) y no admite nulos.

### Atributos [Key] y [PrimaryKey]

[Key] marca una clave primaria simple. Para una clave primaria compuesta en EF Core 8 se puede utilizar [PrimaryKey] sobre el tipo o configurar HasKey con Fluent API.

```
[PrimaryKey(nameof(OrdenFabricacionId), nameof(AleacionId))]
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; }
    public decimal CantidadUtilizada { get; set; }
}
```

CSHARP

[PrimaryKey] declara que las dos propiedades forman conjuntamente la clave primaria. Colocar [Key] por separado en dos propiedades no es la forma correcta de expresar una clave compuesta.

### Atributo [Required]

El atributo [Required] marca una propiedad como requerida. La columna se crea como NOT NULL. Se aplica a propiedades de tipo referencia (string) o a propiedades de tipo valor anulables (int?, DateTime?) que se quieren marcar como requeridas.

CSHARP

```
public class OrdenFabricacion
{
    [Required]
    public string NumeroOrden { get; set; } = string.Empty;

    [Required]
    public string Cliente { get; set; } = string.Empty;
}
```

La primera línea aplica [Required] sobre NumeroOrden. La segunda aplica [Required] sobre Cliente. Ambas propiedades se mapean a columnas NOT NULL.

Error común: si se aplica [Required] a una propiedad de tipo valor no anulable (int, DateTime, bool), el atributo es redundante porque la propiedad ya es requerida por convención. El atributo solo es necesario para propiedades de tipo referencia o de tipo valor anulable.

## Atributo [MaxLength] y [StringLength]

El atributo [MaxLength] establece la longitud máxima de una propiedad de tipo string o byte[]. El atributo [StringLength] hace lo mismo, pero además permite especificar una longitud mínima.

CSHARP

```
public class OrdenFabricacion
{
    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;

    [StringLength(200, MinimumLength = 3)]
    public string Cliente { get; set; } = string.Empty;
}
```

La primera línea aplica [MaxLength(50)] sobre NumeroOrden, que se mapea a nvarchar(50). La segunda línea aplica [StringLength(200, MinimumLength = 3)] sobre Cliente, que se mapea a nvarchar(200) y además valida que la cadena tenga al menos 3 caracteres.

Error común: combinar [MaxLength] y [StringLength] con límites distintos. Ambos pueden aportar metadatos de longitud al modelo y [StringLength] además participa en validación; para evitar una configuración ambigua o difícil de mantener, debe elegirse una única regla de longitud coherente para la propiedad.

## Atributo [Column]

El atributo [Column] establece el nombre y el tipo de la columna a la que se mapea una propiedad. Permite especificar el nombre, el tipo de dato y el orden de la columna.

CSHARP

```
public class OrdenFabricacion
{
    [Column("OrderNumber", TypeName = "varchar(50)")]
    public string NumeroOrden { get; set; } = string.Empty;

    [Column(Order = 2)]
    public string Cliente { get; set; } = string.Empty;
}
```

La primera línea aplica [Column("OrderNumber", TypeName = "varchar(50)")] sobre NumeroOrden. La columna se llama OrderNumber y su tipo es varchar(50). La segunda línea aplica [Column(Order = 2)] sobre Cliente, que establece el orden de la columna en la tabla.

Error común: si se cambia el nombre de la columna con [Column] pero no se actualiza la migración, la base de datos sigue teniendo el nombre anterior. Se debe generar una nueva migración para aplicar el cambio.

## Atributo [Table]

El atributo [Table] establece el nombre de la tabla a la que se mapea una entidad. Se aplica sobre la clase.

CSHARP

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
}
```

La primera línea aplica [Table("OrdenesFabricacion")] sobre la clase. La segunda línea declara la clase. La tabla se llama OrdenesFabricacion. Si no se aplicara el atributo, la tabla se llamaría OrdenFabricacion (el nombre de la clase) o OrdenesFabricacion (el nombre del DbSet), según la convención.

Error común: si se aplica [Table] con un nombre que no coincide con el nombre del DbSet, EF Core usa el nombre del atributo. Esto puede provocar confusión si no se actualiza también el DbSet.

## Atributo [ForeignKey]

El atributo [ForeignKey] especifica la propiedad que actúa como clave foránea. Se aplica sobre la propiedad de navegación o sobre la propiedad de la clave foránea.

```
public class PlanchaAcero
{
    public int Id { get; set; }

    public int OrdenId { get; set; }

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea aplica [ForeignKey(nameof(OrdenId))] sobre la propiedad Orden, indicando que OrdenId es la clave foránea. La segunda línea declara la propiedad de navegación.

También se puede aplicar sobre la propiedad de la clave foránea:

```
public class PlanchaAcero
{
    public int Id { get; set; }

    [ForeignKey(nameof(Orden))]
    public int OrdenId { get; set; }

    public OrdenFabricacion Orden { get; set; } = null!;
}
```

CSHARP

La primera línea aplica [ForeignKey(nameof(Orden))] sobre OrdenId, indicando que Orden es la propiedad de navegación. Ambas formas producen el mismo resultado.

Si [ForeignKey] se coloca sobre una navegación y el nombre indicado no coincide con una propiedad CLR existente, EF Core puede crear una propiedad de sombra con ese nombre para actuar como clave foránea. Esto puede ocultar un error tipográfico; en código de producción conviene usar nameof(...) cuando se pretende referenciar una propiedad real y, si se desea, configurar las advertencias para detectar la creación de propiedades de sombra.

## Atributo [InverseProperty]

El atributo [InverseProperty] especifica la propiedad de navegación inversa cuando hay varias relaciones entre las mismas entidades.

```
public class OrdenFabricacion
{
    public int Id { get; set; }

    [InverseProperty(nameof(PlanchaAcero.Orden))]
    public List<PlanchaAcero> Planchas { get; set; } = new();

    [InverseProperty(nameof(PlanchaAcero.OrdenSecundaria))]
    public List<PlanchaAcero> PlanchasSecundarias { get; set; } = new();
}

public class PlanchaAcero
{
    public int Id { get; set; }
    public int OrdenId { get; set; }
    public int? OrdenSecundariaId { get; set; }

    [ForeignKey(nameof(OrdenId))]
    public OrdenFabricacion Orden { get; set; } = null!;

    [ForeignKey(nameof(OrdenSecundariaId))]
    public OrdenFabricacion? OrdenSecundaria { get; set; }
}
```

CSHARP

La primera línea aplica [InverseProperty] sobre Planchas, indicando que se relaciona con PlanchaAcero.Orden. La segunda línea aplica [InverseProperty] sobre PlanchasSecundarias, indicando que se relaciona con PlanchaAcero.OrdenSecundaria. Sin [InverseProperty], EF Core no sabría qué colección corresponde a qué referencia.

Error común: si se omite [InverseProperty] cuando hay varias propiedades de navegación entre las mismas entidades, EF Core crea relaciones adicionales no deseadas. El atributo es necesario para desambiguar.

### Atributo [NotMapped]

El atributo [NotMapped] excluye una propiedad del mapeo a la base de datos. La propiedad existe en la clase pero no se crea una columna para ella.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;

    [NotMapped]
    public string DescripcionCompleta => $"{NumeroOrden} - {Cliente}";

    public string Cliente { get; set; } = string.Empty;
}
```

CSHARP

La primera línea aplica [NotMapped] sobre DescripcionCompleta, que es una propiedad calculada. La segunda línea declara la propiedad. EF Core no crea una columna para ella.

Error común: asumir que toda propiedad calculada necesita [NotMapped]. Por convención EF Core incluye propiedades públicas con getter y setter; una propiedad que cumple esas convenciones puede mapearse aunque su valor se calcule en la aplicación. [NotMapped] se usa cuando se quiere excluirla explícitamente del modelo. Una propiedad que no cumple las convenciones de inclusión puede quedar ignorada sin necesidad de provocar una excepción.

### Atributo [Index] específico de EF Core

El atributo [Index], definido por EF Core, crea un índice sobre una o varias propiedades. Se aplica sobre la clase, no sobre la propiedad.

```
[Index(nameof(NumeroOrden), IsUnique = true, Name = "IX_OrdenesFabricacion_NumeroOrden")]
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
}
```

CSHARP

La primera línea aplica [Index] sobre la clase, indicando que se cree un índice único sobre NumeroOrden con el nombre IX\_OrdenesFabricacion\_NumeroOrden. La segunda línea declara la clase. La tercera declara la propiedad NumeroOrden.

También se puede aplicar varios atributos [Index] para crear varios índices.

```
[Index(nameof(NumeroOrden), IsUnique = true)]
[Index(nameof(Cliente))]
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public string Cliente { get; set; } = string.Empty;
}
```

CSHARP

La primera línea crea un índice único sobre NumeroOrden. La segunda línea crea un índice no único sobre Cliente. La tercera línea declara la clase.

Error común: si se aplica [Index] con un nombre que ya existe en la base de datos, la migración falla al intentar crear el índice. Se debe usar un nombre único.

### Atributo [Precision]

El atributo [Precision] establece la precisión y la escala de una propiedad de tipo decimal.

```
public class PlanchaAcero
{
    public int Id { get; set; }
```

CSHARP

```
[Precision(18, 3)]
public decimal Peso { get; set; }
}
```

La primera línea aplica [Precision(18, 3)] sobre Peso, que se mapea a decimal(18,3). La segunda línea declara la propiedad.

El atributo [Precision] no se limita a decimal: expresa precisión (y, cuando corresponde, escala) para propiedades cuyo tipo y proveedor admiten esos conceptos, por ejemplo decimal y determinados tipos temporales. La configuración efectiva depende del proveedor de base de datos.

### Atributo [DatabaseGenerated]

El atributo [DatabaseGenerated] especifica cómo se genera el valor de una propiedad. Los valores posibles son None, Identity y Computed.

```
public class OrdenFabricacion
{
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; set; }

    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public DateTime FechaModificacion { get; set; }
}
```

CSHARP

La primera línea aplica [DatabaseGenerated(DatabaseGeneratedOption.Identity)] sobre Id y comunica a EF Core que el valor se genera al insertar. La segunda línea aplica [DatabaseGenerated(DatabaseGeneratedOption.Computed)] sobre FechaModificacion y comunica a EF Core que el valor se genera al insertar o actualizar.

Es importante distinguir la semántica de EF Core del mecanismo de base de datos:

[DatabaseGenerated(DatabaseGeneratedOption.Computed)] no crea por sí solo una fórmula ni un mecanismo automático para actualizar un DateTime. En SQL Server, una marca temporal de creación suele configurarse con un valor por defecto como GETDATE() o GETUTCDATE(); una marca de última modificación requiere un mecanismo explícito, por ejemplo un trigger. El atributo indica a EF Core que el valor lo genera la base de datos, pero la estrategia concreta debe existir realmente en el proveedor.

Error común: asumir que [DatabaseGenerated] crea automáticamente el mecanismo de generación en SQL Server para cualquier tipo. La generación depende del proveedor y de la configuración de la columna; para DateTime debe configurarse expresamente el comportamiento deseado.

### Atributo [ConcurrencyCheck] y [Timestamp]

El atributo [ConcurrencyCheck] marca una propiedad como token de concurrencia. El atributo [Timestamp] marca una propiedad de tipo byte[] como token de concurrencia y además indica que la base de datos genera el valor en cada actualización.

```
public class OrdenFabricacion
{
    public int Id { get; set; }

    [ConcurrencyCheck]
    public string Estado { get; set; } = string.Empty;

    [Timestamp]
    public byte[]? RowVersion { get; set; }
}
```

CSHARP

La primera línea aplica [ConcurrencyCheck] sobre Estado, que se usará para detectar conflictos de concurrencia. La segunda línea aplica [Timestamp] sobre RowVersion, que se genera automáticamente en cada actualización.

Error común: si se aplica [Timestamp] a una propiedad que no es de tipo byte[], EF Core lanza una excepción al construir el modelo.

### Atributo [BackingField]

El atributo [BackingField] especifica el campo de respaldo que EF Core debe usar para una propiedad. Se aplica sobre la propiedad.

CSHARP

```
public class OrdenFabricacion
{
    private string _numeroOrden = string.Empty;

    [BackingField(nameof(_numeroOrden))]
    public string NumeroOrden
    {
        get => _numeroOrden;
        set => _numeroOrden = value;
    }
}
```

La primera línea declara el campo de respaldo. La segunda línea aplica [BackingField] sobre la propiedad, indicando que EF Core use el campo \_numeroOrden para leer y escribir el valor.

Error común: si el campo de respaldo no existe o no tiene el nombre indicado, EF Core lanza una excepción al construir el modelo.

## Atributo [Comment]

El atributo [Comment] añade un comentario a la columna en la base de datos. Se aplica sobre la propiedad.

CSHARP

```
public class OrdenFabricacion
{
    [Comment("Número único de la orden en formato OF-YYYY-NNNN")]
    public string NumeroOrden { get; set; } = string.Empty;
}
```

La primera línea aplica [Comment] sobre NumeroOrden. El comentario se incluye en el script de creación de la tabla.

Error común: si el motor de base de datos no soporta comentarios en columnas, EF Core ignora el atributo. SQL Server sí los soporta.

## Prioridad de configuración

EF Core aplica la configuración en el siguiente orden de prioridad: primero las convenciones, después las Data Annotations y finalmente la Fluent API. Esto significa que si una propiedad está configurada con Data Annotations y también con Fluent API, prevalece la Fluent API.

CSHARP

```
// Data Annotations
[MaxLength(50)]
public string NumeroOrden { get; set; } = string.Empty;

// Fluent API
modelBuilder.Entity<OrdenFabricacion>()
    .Property(o => o.NumeroOrden)
    .HasMaxLength(100);
```

La primera línea aplica [MaxLength(50)]. La segunda línea configura la propiedad con HasMaxLength(100) en Fluent API. La longitud final es 100 porque la Fluent API tiene prioridad.

Error común: si se configura la misma propiedad con Data Annotations y Fluent API con valores distintos, la Fluent API prevalece y la configuración de Data Annotations se ignora. Esto puede provocar confusión si no se revisa la configuración de Fluent API.

Cuándo usar Data Annotations y cuándo Fluent API

Las Data Annotations son adecuadas para configuraciones simples y locales: claves primarias, longitudes máximas, propiedades requeridas, nombres de columna y tabla. La Fluent API es adecuada para configuraciones complejas: relaciones con múltiples propiedades de navegación, índices compuestos, filtros globales, comportamientos de eliminación y configuración de entidades intermedias.

En proyectos grandes, se suele usar Fluent API de forma exclusiva para mantener toda la configuración en un solo lugar. En proyectos pequeños o en prototipos, las Data Annotations son más rápidas de escribir y mantener.

Coexistencia de Data Annotations y Fluent API

Ambas formas de configuración pueden coexistir en el mismo modelo. EF Core aplica primero las convenciones, después las Data Annotations y finalmente la Fluent API. La Fluent API puede sobrescribir la configuración de Data Annotations.

```
[Table("OrdenesFabricacion")]
public class OrdenFabricacion
{
    [Key]
    public int Id { get; set; }

    [MaxLength(50)]
    public string NumeroOrden { get; set; } = string.Empty;
}

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>()
        .Property(o => o.NumeroOrden)
        .HasMaxLength(100)
        .IsRequired();
}
```

La primera línea aplica [Table("OrdenesFabricacion")] sobre la clase. La segunda línea aplica [Key] sobre Id. La tercera línea aplica [MaxLength(50)] sobre NumeroOrden. La cuarta línea configura la propiedad NumeroOrden con HasMaxLength(100) y IsRequired() en Fluent API. La longitud final es 100 porque la Fluent API tiene prioridad sobre [MaxLength(50)].

Error común: si se configura la misma propiedad con valores contradictorios en Data Annotations y Fluent API, la Fluent API prevalece silenciosamente. Se debe revisar la configuración de Fluent API para evitar conflictos.

## El proyecto AceriaData

En el proyecto AceriaData, se aplican Data Annotations a las entidades OrdenFabricacion, PlanchaAcero, Aleacion, EstadoOrden, DetalleOrden, CertificadoCalidad y OrdenAleacion. Se configuran claves primarias, longitudes máximas, propiedades requeridas, nombres de columna, índices y claves foráneas. La configuración de Fluent API existente se mantiene para las relaciones y configuraciones complejas, mientras que las Data Annotations se usan para las configuraciones simples.

## Resumen de la teoría

Las Data Annotations son atributos que se aplican sobre clases y propiedades.

[Key] marca la clave primaria.

[Required] marca una propiedad como requerida.

[MaxLength] y [StringLength] establecen la longitud máxima de una cadena.

[Column] establece el nombre y el tipo de la columna.

[Table] establece el nombre de la tabla.

[ForeignKey] especifica la clave foránea.

[InverseProperty] desambigua las propiedades de navegación.

[NotMapped] excluye una propiedad del mapeo.

[Index] crea un índice.

[Precision] establece la precisión decimal.

[DatabaseGenerated] especifica cómo se genera el valor.

[ConcurrencyCheck] y [Timestamp] marcan tokens de concurrencia.

[BackingField] especifica el campo de respaldo.

[Comment] añade un comentario a la columna.

La prioridad es: convenciones < Data Annotations < Fluent API.

Data Annotations y Fluent API pueden coexistir.

## Punto 2.7 - Configuración mediante Fluent API

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se profundiza en la sintaxis y prioridad de Fluent API sobre el estado acumulado de AceriaData, configurando propiedades y relaciones sin adelantar en el proyecto las claves de 2.8, los índices/restricciones de 2.9 ni los filtros globales de 2.10.

### Objetivos de aprendizaje

Comprender qué es la Fluent API y cómo prevalece sobre convenciones y Data Annotations.

Configurar de forma explícita propiedades escalares, valores por defecto y tipos de columna.

Configurar propiedades de navegación y relaciones desde Fluent API.

Comprender cómo se organiza la configuración de entidades intermedias.

Reconocer table splitting y owned types como capacidades de modelado avanzado.

Identificar qué capacidades de Fluent API se desarrollarán específicamente en 2.8, 2.9 y 2.10.

Aplicar al estado 2.7 únicamente la configuración correspondiente a este punto, manteniendo la progresión acumulativa de AceriaData.

### Teoría

#### Qué es la Fluent API

La Fluent API es el mecanismo de configuración imperativa de EF Core. Se escribe en el método `OnModelCreating` del `DbContext` y recibe un objeto `ModelBuilder` que expone métodos encadenados para configurar entidades, propiedades, relaciones, índices y restricciones. Su nombre proviene del estilo de programación "fluido", en el que los métodos se encadenan uno tras otro formando una frase legible.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<OrdenFabricacion>(entity =>
    {
        entity.ToTable("OrdenesFabricacion");
        entity.HasKey(o => o.Id);

        entity.Property(o => o.NumeroOrden)
            .IsRequired()
            .HasMaxLength(50)
            .HasComment("Número único de la orden");

        entity.HasIndex(o => o.NumeroOrden)
            .IsUnique()
            .HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");
    });
}
```

CSHARP

La primera línea declara el método `OnModelCreating`. La segunda línea selecciona la entidad `OrdenFabricacion` con el método `Entity<T>`. La tercera línea establece el nombre de la tabla. La cuarta línea declara la clave primaria. La quinta línea selecciona la propiedad `NumeroOrden`. La sexta línea la marca como requerida. La séptima línea establece la longitud máxima. La octava línea añade un comentario. La novena línea crea un índice. La décima línea lo marca como único. La undécima línea establece el nombre del índice.

La Fluent API tiene varias ventajas sobre las Data Annotations. La primera es que permite configurar todo el modelo desde un solo lugar, sin dispersar atributos por las clases. La segunda es que expone métodos para configuraciones que no tienen equivalente en Data Annotations, como los índices compuestos, las restricciones CHECK y los filtros globales. La tercera es que tiene prioridad sobre las Data Annotations, lo que permite sobrescribir configuraciones sin modificar las clases.

#### Vista previa: índices con Fluent API (se desarrollan en 2.9)

Un índice simple se crea con el método `HasIndex` sobre la entidad. Se puede marcar como único con `IsUnique` y nombrar con `HasDatabaseName`.

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasIndex(o => o.NumeroOrden)
        .IsUnique()
        .HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");
});
```

La primera línea selecciona la entidad. La segunda crea un índice sobre NumeroOrden. La tercera lo marca como único. La cuarta establece el nombre del índice. El índice se crea en la base de datos con la sentencia CREATE UNIQUE INDEX.

Límites de este punto

Fluent API también permite configurar claves alternativas, índices, restricciones y filtros globales. Para mantener la progresión del módulo, las claves se desarrollan en 2.8, los índices y restricciones en 2.9 y los filtros globales en 2.10. Aquí se estudia la sintaxis, la prioridad y la configuración explícita de propiedades y relaciones.

## Configurar table splitting

El table splitting es una técnica que permite mapear dos entidades a la misma tabla. Es útil cuando se quiere separar una entidad en dos clases pero mantener una sola tabla en la base de datos. Las dos entidades comparten la misma clave primaria.

CSHARP

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(o => o.Id);

    entity.HasOne(o => o.Detalle)
        .WithOne()
        .HasForeignKey<DetalleOrden>(d => d.Id);
});

modelBuilder.Entity<DetalleOrden>(entity =>
{
    entity.ToTable("OrdenesFabricacion");
    entity.HasKey(d => d.Id);
});
```

La primera línea configura la entidad OrdenFabricacion. La segunda establece la tabla. La tercera declara la clave primaria. La cuarta configura la relación con DetalleOrden. La quinta indica que es uno a uno. La sexta especifica la clave foránea, que también es la clave primaria. La séptima configura la entidad DetalleOrden. La octava establece la misma tabla. La novena declara la clave primaria, que coincide con la de OrdenFabricacion. Las dos entidades comparten la tabla OrdenesFabricacion.

## Configurar owned types

Los owned types son tipos que pertenecen a una entidad y no tienen identidad propia. Se mapean a las mismas columnas de la entidad propietaria o a una tabla separada. Se configuran con OwnsOne u OwnsMany.

CSHARP

```
public class Direccion
{
    public string Calle { get; set; } = string.Empty;
    public string Ciudad { get; set; } = string.Empty;
    public stringCodigoPostal { get; set; } = string.Empty;
}

public class Cliente
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public Direccion Direccion { get; set; } = null!;
}

modelBuilder.Entity<Cliente>(entity =>
{
    entity.OwnsOne(c => c.Direccion, direccion =>
    {
        direccion.Property(d => d.Calle).HasMaxLength(200);
        direccion.Property(d => d.Ciudad).HasMaxLength(100);
        direccion.Property(d => d.CodigoPostal).HasMaxLength(10);
    });
});
```

La primera línea declara la clase `Direccion` como owned type. La segunda declara la clase `Cliente` con una propiedad `Direccion`. La tercera línea selecciona la entidad `Cliente`. La cuarta configura el owned type `Direccion`. La quinta configura la propiedad `Calle`. La sexta configura la propiedad `Ciudad`. La séptima configura la propiedad `CodigoPostal`. Las columnas del owned type se incluyen en la tabla `Clientes` con el prefijo `Direccion_`.

## Configurar la sensibilidad a mayúsculas

La sensibilidad a mayúsculas se configura con `UseCollation`. Es útil cuando se quiere que las comparaciones de cadenas sean sensibles o insensibles a mayúsculas.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.Cliente)
        .UseCollation("SQL_Latin1_General_CP1_CS_AS");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda selecciona la propiedad `Cliente`. La tercera establece explícitamente la collation `SQL_Latin1_General_CP1_CS_AS`, que distingue mayúsculas y minúsculas. No debe asumirse una collation predeterminada concreta para todas las instalaciones de SQL Server: la collation efectiva depende de la instancia, la base de datos y, si se configura, la propia columna.

## Configurar el orden de las columnas

El orden de las columnas se configura con `HasColumnOrder`. Es útil cuando se quiere que las columnas aparezcan en un orden concreto en la tabla.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.Id).HasColumnOrder(0);
    entity.Property(o => o.NumeroOrden).HasColumnOrder(1);
    entity.Property(o => o.Cliente).HasColumnOrder(2);
    entity.Property(o => o.FechaCreacion).HasColumnOrder(3);
});
```

CSHARP

La primera línea selecciona la entidad. La segunda establece el orden 0 para `Id`. La tercera establece el orden 1 para `NumeroOrden`. La cuarta establece el orden 2 para `Cliente`. La quinta establece el orden 3 para `FechaCreacion`.

## Configurar el tipo de columna

El tipo de columna se configura con `HasColumnType`. Es útil cuando se quiere usar un tipo específico del motor que no coincide con el mapeo por defecto.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.NumeroOrden)
        .HasColumnType("varchar(50)");

    entity.Property(o => o.Observaciones)
        .HasColumnType("nvarchar(max)");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda selecciona la propiedad `NumeroOrden`. La tercera establece el tipo `varchar(50)`, que es no Unicode. La cuarta selecciona `Observaciones`. La quinta usa `nvarchar(max)` para texto Unicode de gran tamaño. En SQL Server no se recomienda usar el tipo `text` en desarrollos nuevos porque está obsoleto; para texto largo deben preferirse `varchar(max)` o `nvarchar(max)`, según las necesidades de Unicode.

## Configurar la propagación de claves

La propagación de claves se configura con `ValueGeneratedOnAdd`, `ValueGeneratedOnAddOrUpdate` y `ValueGeneratedNever`. El primero indica que la base de datos genera el valor al insertar. El segundo indica que la base de datos genera el valor al insertar o actualizar. El tercero indica que la base de datos nunca genera el valor.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.Id)
        .ValueGeneratedOnAdd();

    entity.Property(o => o.FechaModificacion)
        .ValueGeneratedOnAddOrUpdate();
});
```

CSHARP

```
entity.Property(o => o.NumeroOrden)
    .ValueGeneratedNever();
});
```

La primera línea selecciona la entidad. La segunda selecciona la propiedad Id. La tercera indica a EF Core que el valor se genera al insertar. La cuarta selecciona FechaModificacion y la quinta la marca como generada al insertar o actualizar. Estas llamadas describen a EF Core cuándo esperar un valor generado, pero no crean necesariamente el mecanismo de generación en la base de datos. En SQL Server, un DateTime de última modificación necesita una estrategia explícita, como un trigger. La sexta selecciona NumeroOrden y la séptima desactiva la generación de valores para esa propiedad.

### Vista previa: claves alternativas con Fluent API (se desarrollan en 2.8)

Las propiedades alternativas son propiedades que no forman parte de la clave primaria pero que tienen un valor único. Se configuran con HasAlternateKey.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasAlternateKey(o => o.NumeroOrden);
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara NumeroOrden como clave alternativa. La clave alternativa crea un índice único y puede ser referenciada por claves foráneas.

### Configurar la exclusión de una entidad del modelo

Una entidad se excluye del modelo con Ignore. Es útil cuando se quiere que una clase no se mapee a una tabla.

```
modelBuilder.Ignore<EntidadNoMapeada>();
```

CSHARP

La primera línea excluye la entidad EntidadNoMapeada del modelo. La entidad no se mapea a ninguna tabla y no se incluye en las migraciones.

### Configurar una propiedad como no mapeada

Una propiedad se excluye del mapeo con Ignore sobre la entidad.

```
modelBuilder.Entity<OrdenFabricacion>()
    .Ignore(o => o.Resumen);
```

CSHARP

La primera línea selecciona la entidad. La segunda excluye la propiedad Resumen del mapeo. La propiedad existe en la clase pero no se crea una columna para ella.

### Configurar el nombre de la clave foránea

El nombre de la clave foránea se configura con HasConstraintName.

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(p => p.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(p => p.OrdenId)
        .HasConstraintName("FK_PlanchasAcero_OrdenesFabricacion");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda configura la relación. La tercera indica que una orden tiene muchas planchas. La cuarta especifica la clave foránea. La quinta establece el nombre de la restricción.

## El proyecto AceriaData

En el estado 2.7, AceriaData centraliza configuración explícita mediante Fluent API y conserva el modelo acumulado hasta 2.6. Este punto trabaja sintaxis, propiedades, valores por defecto y relaciones; no incorpora todavía las claves alternativas de 2.8, los índices/restricciones específicos de 2.9 ni los filtros globales de 2.10. La separación permite que cada estado del proyecto introduzca sólo el concepto que le corresponde.

## Resumen de la teoría

La Fluent API es el mecanismo de configuración imperativa de EF Core.

Se escribe en el método `OnModelCreating` del `DbContext`.

También permite configurar índices y restricciones, capacidades que en este curso se desarrollan específicamente en 2.9.

Permite configurar valores por defecto con expresiones SQL.

Permite configurar la tabla intermedia con `UsingEntity`.

Permite configurar table splitting.

Permite configurar owned types.

Permite configurar la collation de las columnas.

Permite configurar el orden de las columnas.

Permite configurar el tipo de columna.

Permite configurar la propagación de claves.

También permite configurar claves alternativas, que en este curso se desarrollan específicamente en 2.8.

Permite excluir entidades y propiedades del modelo.

Permite configurar el nombre de las restricciones.

## Punto 2.8 - Claves primarias, claves alternativas y claves compuestas

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configuran explícitamente las claves primarias, las claves alternativas y las claves compuestas del proyecto `AceriaData`, definiendo la unicidad de los identificadores y las restricciones de integridad en el modelo.

### Objetivos de aprendizaje

Comprender qué es una clave primaria y cómo se configura.

Diferenciar entre clave primaria simple y clave primaria compuesta.

Comprender qué es una clave alternativa y cuándo usarla.

Configurar claves primarias y alternativas con Fluent API.

Configurar claves primarias compuestas en entidades intermedias.

Comprender el impacto de las claves en los índices y las restricciones.

Aplicar estas configuraciones al proyecto `AceriaData`.

## Teoría

### Qué es una clave primaria

Una clave primaria es el atributo o conjunto de atributos que identifican de forma única a cada fila de una tabla. En EF Core, cada entidad debe tener una clave primaria. Por convención, EF Core detecta la clave primaria por el nombre `Id` o `<Clase>Id`. Si no encuentra ninguna propiedad que cumpla el patrón, lanza una excepción al construir el modelo.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
}
```

CSHARP

La primera línea declara la propiedad `Id` de tipo `int`. EF Core la detecta como clave primaria por convención. La segunda línea declara `NumeroOrden`, que no es clave primaria.

La clave primaria se mapea a la restricción PRIMARY KEY de la tabla. Esta restricción garantiza que no haya dos filas con el mismo valor de clave y que el valor no sea nulo. Además, SQL Server crea un índice agrupado (clustered index) sobre la clave primaria por defecto, lo que determina el orden físico de las filas en la tabla.

## Configurar la clave primaria con Fluent API

La clave primaria se configura con el método `HasKey` sobre la entidad. Se puede configurar una clave primaria simple o compuesta.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasKey(o => o.Id);
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara `Id` como clave primaria. Si la clave primaria ya se detecta por convención, esta configuración es redundante pero explícita.

## Configurar la clave primaria con nombre personalizado

Si la clave primaria no sigue la convención, se puede configurar con `HasKey`. Además, se puede establecer el nombre de la restricción con `HasName`.

```
public class OrdenFabricacion
{
    public int CodigoOrden { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
}

modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasKey(o => o.CodigoOrden)
        .HasName("PK_OrdenesFabricacion");
});
```

CSHARP

La primera línea declara la propiedad `CodigoOrden` como clave primaria. La segunda línea configura la clave primaria con `HasKey`. La tercera línea establece el nombre de la restricción `PK_OrdenesFabricacion`.

Error común: si se configura `HasKey` sobre una propiedad que no existe, EF Core lanza una excepción al construir el modelo. La expresión debe apuntar a una propiedad válida.

### Clave primaria compuesta

Una clave primaria compuesta está formada por dos o más propiedades. Se usa cuando ninguna propiedad por sí sola identifica de forma única una fila. En el proyecto `AceriaData`, la entidad `OrdenAleacion` tiene una clave primaria compuesta formada por `OrdenFabricacionId` y `AleacionId`.

```
public class OrdenAleacion
{
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public DateTime FechaAsignacion { get; set; }
}
```

CSHARP

La primera línea declara `OrdenFabricacionId`. La segunda línea declara `AleacionId`. Ninguna de las dos por sí sola identifica de forma única una fila, pero la combinación de ambas sí.

La clave primaria compuesta se configura con `HasKey` pasando una expresión que devuelve un objeto anónimo con las propiedades que la forman.

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.HasKey(oa => new { oa.OrdenFabricacionId, oa.AleacionId });
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara la clave primaria compuesta formada por `OrdenFabricacionId` y `AleacionId`. EF Core crea una restricción PRIMARY KEY sobre ambas columnas y un índice agrupado compuesto.

El orden de las propiedades en una clave compuesta importa para la definición de la propia clave y para las claves foráneas que la referencian: el orden y los tipos deben coincidir. En SQL Server, si esa clave está respaldada por el índice agrupado predeterminado, el orden de sus propiedades también determina el orden de las columnas clave del índice.

## Configurar el orden de las columnas de la clave compuesta

El orden de las columnas en la clave compuesta se puede configurar con `HasKey` pasando las propiedades en el orden deseado. También se puede configurar el orden de las columnas con `HasColumnOrder` en cada propiedad.

```
modelBuilder.Entity<OrdenAleacion>(entity =>
{
    entity.HasKey(oa => new { oa.OrdenFabricacionId, oa.AleacionId });

    entity.Property(oa => oa.OrdenFabricacionId)
        .HasColumnOrder(1);

    entity.Property(oa => oa.AleacionId)
        .HasColumnOrder(2);
});
```

CSHARP

La primera línea declara la clave primaria compuesta y fija el orden lógico de las propiedades de la clave. Las llamadas `HasColumnOrder` configuran, por separado, el orden físico de las columnas cuando la tabla se crea mediante Migrations. El orden de una clave compuesta no tiene por qué coincidir con el orden general de las columnas de la tabla.

Error común: confundir el orden de propiedades de una clave compuesta con `HasColumnOrder`. Cambiar el orden de la clave modifica la definición de la restricción y de las relaciones que la referencian; `HasColumnOrder` sólo controla la posición de las columnas al crear la tabla y, en general, no sirve para reordenar columnas de una tabla ya existente.

### Clave alternativa

Una clave alternativa es una propiedad o conjunto de propiedades que identifican de forma única una fila pero que no son la clave primaria. Se usa cuando hay más de un identificador único en la entidad. En el proyecto *AceriaData*, el `NumeroOrden` es un identificador único de la orden, pero la clave primaria es `Id`. El `NumeroOrden` se configura como clave alternativa.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasAlternateKey(o => o.NumeroOrden)
        .HasName("AK_OrdenesFabricacion_NumeroOrden");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara `NumeroOrden` como clave alternativa con el nombre `AK_OrdenesFabricacion_NumeroOrden`. EF Core crea un índice único sobre la columna `NumeroOrden` y una restricción `UNIQUE`.

La diferencia entre una clave alternativa y un índice único es que la clave alternativa puede ser referenciada por una clave foránea. Esto permite que otras entidades se relacionen con la entidad principal a través de la clave alternativa en lugar de la clave primaria.

```
public class PlanchaAcero
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public OrdenFabricacion Orden { get; set; } = null!;
}

modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(p => p.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(p => p.NumeroOrden)
        .HasPrincipalKey(o => o.NumeroOrden);
});
```

CSHARP

La primera línea declara la clave foránea `NumeroOrden` en `PlanchaAcero`. La segunda línea configura la relación con `OrdenFabricacion`. La tercera línea indica que cada orden tiene muchas planchas. La cuarta línea especifica la clave foránea. La quinta línea especifica la clave principal, que es la clave alternativa `NumeroOrden`. La relación se establece a través de la clave alternativa en lugar de la clave primaria.

Al configurar una relación hacia una propiedad que no es la clave primaria, `HasPrincipalKey` establece esa propiedad como clave principal de la relación e introduce la clave alternativa necesaria si aún no existe. `HasAlternateKey` puede declararse explícitamente cuando se quiere configurar esa clave por separado, por ejemplo para asignarle un nombre de restricción.

#### Clave alternativa compuesta

Una clave alternativa también puede ser compuesta. Se configura con `HasAlternateKey` pasando una expresión que devuelve un objeto anónimo.

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.HasAlternateKey(a => new { a.Nombre, a.Codigo })
        .HasName("AK_Aleaciones_Nombre_Codigo");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara la clave alternativa compuesta formada por `Nombre` y `Codigo`. La tercera línea establece el nombre de la restricción. EF Core crea un índice único compuesto sobre ambas columnas.

Error común: si la combinación de propiedades no es única en los datos existentes, la migración falla al crear el índice único. Se debe verificar que los datos cumplen la restricción antes de aplicarla.

#### Clave foránea

Una clave foránea es una propiedad o conjunto de propiedades que referencian la clave primaria o alternativa de otra entidad. Se configura con `HasForeignKey` en la relación. En el proyecto `AceriaData`, la entidad `PlanchaAcero` tiene una clave foránea `OrdenId` que referencia la clave primaria `Id` de `OrdenFabricacion`.

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.HasOne(p => p.Orden)
        .WithMany(o => o.Planchas)
        .HasForeignKey(p => p.OrdenId)
        .HasConstraintName("FK_PlanchasAcero_OrdenesFabricacion");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda configura la relación. La tercera indica que cada orden tiene muchas planchas. La cuarta especifica la clave foránea `OrdenId`. La quinta establece el nombre de la restricción `FK_PlanchasAcero_OrdenesFabricacion`.

#### Clave foránea compuesta

Una clave foránea compuesta está formada por dos o más propiedades. Se usa cuando la entidad dependiente referencia una clave primaria compuesta. En el proyecto `AceriaData`, si se quisiera referenciar la entidad `OrdenAleacion` desde otra entidad, la clave foránea sería compuesta.

```
public class ConsumoAleacion
{
    public int Id { get; set; }
    public int OrdenFabricacionId { get; set; }
    public int AleacionId { get; set; }
    public decimal CantidadConsumida { get; set; }
    public OrdenAleacion OrdenAleacion { get; set; } = null!;
}

modelBuilder.Entity<ConsumoAleacion>(entity =>
{
    entity.HasOne(c => c.OrdenAleacion)
        .WithMany()
        .HasForeignKey(c => new { c.OrdenFabricacionId, c.AleacionId })
        .HasConstraintName("FK_ConsumosAleaciones_OrdenesAleaciones");
});
```

CSHARP

La primera línea declara la entidad `ConsumoAleacion`. La segunda línea declara la clave foránea `OrdenFabricacionId`. La tercera línea declara la clave foránea `AleacionId`. La cuarta línea declara la propiedad de navegación. La quinta línea configura la relación. La sexta línea especifica la clave foránea compuesta. La séptima línea establece el nombre de la restricción.

Error común: si la clave foránea compuesta no coincide con la clave primaria compuesta de la entidad principal, EF Core lanza una excepción al construir el modelo. El orden y los tipos de las propiedades deben coincidir.

#### La relación entre clave primaria, índice y restricción

En SQL Server, la clave primaria se implementa como una restricción `PRIMARY KEY` y, por defecto, queda respaldada por un índice agrupado único. Una clave alternativa se implementa como una restricción `UNIQUE` y normalmente

queda respaldada por un índice único no agrupado. La clave foránea se implementa como una restricción FOREIGN KEY que garantiza la integridad referencial. El tipo agrupado/no agrupado puede configurarse explícitamente con las extensiones del proveedor de SQL Server.

SQL

```
ALTER TABLE [OrdenesFabricacion]
ADD CONSTRAINT [PK_OrdenesFabricacion] PRIMARY KEY ([Id]);

ALTER TABLE [OrdenesFabricacion]
ADD CONSTRAINT [AK_OrdenesFabricacion_NumeroOrden] UNIQUE ([NumeroOrden]);

ALTER TABLE [PlanchasAcero]
ADD CONSTRAINT [FK_PlanchasAcero_OrdenesFabricacion]
FOREIGN KEY ([OrdenId]) REFERENCES [OrdenesFabricacion] ([Id]);
```

La primera sentencia crea la restricción de clave primaria sobre Id. La segunda crea la restricción de clave alternativa sobre NumeroOrden. La tercera crea la restricción de clave foránea sobre OrdenId.

#### Clave primaria vs clave alternativa

La clave primaria es el identificador principal de la entidad. Se usa en las relaciones y en las referencias internas. La clave alternativa es un identificador secundario que también es único. Se usa cuando hay un identificador natural que no es la clave primaria, como el número de orden o el código de aleación.

En el proyecto AceriaData, la clave primaria de OrdenFabricacion es Id. La clave alternativa es NumeroOrden. La clave primaria de Aleacion es Id. La clave alternativa es Codigo. La clave primaria de OrdenAleacion es compuesta: OrdenFabricacionId y AleacionId.

#### Impacto en el rendimiento

Las claves primarias y alternativas crean índices que aceleran las búsquedas. Las claves foráneas crean índices que aceleran las consultas de relación. Sin embargo, cada índice adicional ralentiza las inserciones y actualizaciones porque la base de datos debe mantener el índice actualizado. Se deben crear solo los índices necesarios para las consultas frecuentes.

La clave primaria se crea automáticamente en todas las tablas. La clave alternativa se crea solo cuando se necesita. La clave foránea se crea automáticamente en las relaciones. Los índices adicionales se crean solo cuando las consultas lo requieren.

## El proyecto AceriaData

En el proyecto AceriaData, las claves se configuran en este punto. Se define la clave primaria de cada entidad, se configuran las claves alternativas para los identificadores naturales y se configura la clave primaria compuesta de la entidad intermedia OrdenAleacion. Se establecen los nombres de las restricciones para que las migraciones sean legibles. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

## Resumen de la teoría

La clave primaria identifica de forma única cada fila.

Se detecta por convención con el nombre Id o <Clase>Id.

Se configura con HasKey.

Puede ser simple o compuesta.

La clave alternativa es un identificador único secundario.

Se configura con HasAlternateKey.

La clave foránea referencia la clave primaria o alternativa de otra entidad.

Se configura con HasForeignKey.

Puede ser simple o compuesta.

La clave primaria crea un índice agrupado único.

La clave alternativa crea un índice no agrupado único.

La clave foránea crea una restricción de integridad referencial.

Cada índice adicional ralentiza las inserciones y actualizaciones.

En el proyecto AceriaData se configuran claves primarias, alternativas y compuestas.

## Punto 2.9 - Índices y restricciones

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se configuran explícitamente los índices y las restricciones del proyecto AceriaData, incluyendo índices únicos, índices compuestos, índices filtrados, columnas incluidas, restricciones CHECK, restricciones DEFAULT y restricciones UNIQUE.

### Objetivos de aprendizaje

Comprender qué es un índice y por qué mejora el rendimiento de las consultas.

Diferenciar entre índice agrupado y no agrupado.

Configurar índices únicos y no únicos con Fluent API.

Configurar índices compuestos y su orden de columnas.

Configurar índices filtrados con HasFilter.

Configurar columnas incluidas con IncludeProperties.

Configurar restricciones CHECK, DEFAULT y UNIQUE.

Comprender el impacto de los índices en el rendimiento de escritura.

Aplicar estas configuraciones al proyecto AceriaData.

### Teoría

#### Qué es un índice

Un índice es una estructura de datos auxiliar que la base de datos mantiene junto a la tabla para acelerar las búsquedas. Sin índice, la base de datos debe recorrer todas las filas de la tabla para encontrar las que cumplen una condición. Este recorrido se conoce como table scan y es muy costoso en tablas grandes. Con índice, la base de datos consulta la estructura auxiliar y localiza las filas directamente, sin recorrer toda la tabla.

```
SELECT * FROM OrdenesFabricacion WHERE Cliente = 'Constructora del Norte';
```

SQL

Sin índice sobre Cliente, SQL Server recorre todas las filas de OrdenesFabricacion hasta encontrar las que cumplen la condición. Con un índice sobre Cliente, SQL Server consulta el índice, localiza las filas y las devuelve directamente. La diferencia de rendimiento puede ser de varios órdenes de magnitud en tablas con millones de filas.

#### Índice agrupado y no agrupado

SQL Server soporta dos tipos de índices: agrupados y no agrupados. El índice agrupado determina el orden físico de las filas en la tabla. Solo puede haber un índice agrupado por tabla. Por defecto, la clave primaria se implementa como un índice agrupado. El índice no agrupado es una estructura separada que contiene los valores de las columnas indexadas y un puntero a la fila correspondiente. Puede haber varios índices no agrupados por tabla.

```
-- Índice agrupado (creado por la clave primaria)
CREATE CLUSTERED INDEX PK_OrdenesFabricacion ON OrdenesFabricacion(Id);

-- Índice no agrupado
CREATE NONCLUSTERED INDEX IX_OrdenesFabricacion_Cliente ON OrdenesFabricacion(Cliente);
```

SQL

La primera sentencia crea el índice agrupado sobre Id. La segunda crea el índice no agrupado sobre Cliente. El índice agrupado ordena las filas físicamente por Id. El índice no agrupado contiene los valores de Cliente y un puntero a la fila.

#### Índice único y no único

Un índice único garantiza que no haya dos filas con el mismo valor en las columnas indexadas. Se usa para hacer cumplir restricciones de unicidad. Un índice no único permite duplicados y se usa para acelerar búsquedas.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasIndex(o => o.NumeroOrden)
```

CSHARP

```
.IsUnique()
.HasDatabaseName("IX_OrdenesFabricacion_NumeroOrden");

entity.HasIndex(o => o.Cliente)
.HasDatabaseName("IX_OrdenesFabricacion_Cliente");
});
```

La primera línea selecciona la entidad. La segunda crea un índice sobre NumeroOrden. La tercera lo marca como único. La cuarta establece el nombre. La quinta crea un índice no único sobre Cliente. La sexta establece el nombre. El primer índice impide duplicados en NumeroOrden. El segundo acelera las búsquedas por Cliente.

Error común: si se crea un índice único sobre una columna que ya tiene duplicados, la migración falla al crear el índice. Se deben limpiar los datos antes de aplicar la migración.

### Índice compuesto

Un índice compuesto está formado por dos o más columnas. Se usa cuando las consultas filtran u ordenan por varias columnas a la vez. El orden de las columnas en el índice importa: la primera columna es la más significativa.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasIndex(o => new { o.Cliente, o.FechaCreacion })
        .HasDatabaseName("IX_OrdenesFabricacion_Cliente_FechaCreacion");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda crea un índice compuesto sobre Cliente y FechaCreacion. La tercera establece el nombre. Este índice acelera las consultas que filtran por Cliente y ordenan por FechaCreacion. También acelera las consultas que filtran solo por Cliente, porque la primera columna del índice es Cliente. Pero no acelera las consultas que filtran solo por FechaCreacion, porque la primera columna del índice es Cliente.

Error común: elegir el orden de un índice compuesto sin estudiar las consultas reales. En general, un índice sobre (A, B) puede ayudar a consultas que usan el prefijo A y a consultas que usan A y B, pero no suele resolver de la misma forma una consulta que filtra sólo por B. El orden debe decidirse según los predicados, ordenaciones y patrones de acceso, no mediante una regla universal de «columna más selectiva primero».

### Índice filtrado

Un índice filtrado es un índice que solo incluye un subconjunto de filas de la tabla. Se configura con HasFilter y una condición SQL. Es útil cuando las consultas siempre filtran por un valor concreto.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasIndex(o => o.FechaEntrega)
        .HasFilter("[Estado] = 'Pendiente'")
        .HasDatabaseName("IX_OrdenesFabricacion_FechaEntrega_Pendientes");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda crea un índice sobre FechaEntrega. La tercera establece el filtro [Estado] = 'Pendiente'. La cuarta establece el nombre. El índice solo contiene las filas cuyo estado es "Pendiente". Las consultas que buscan órdenes pendientes por fecha de entrega se benefician del índice. Las consultas que buscan todas las órdenes, independientemente del estado, no usan este índice.

Los índices filtrados son más pequeños que los índices completos, ocupan menos espacio y se actualizan más rápido. Son especialmente útiles cuando la condición del filtro excluye la mayoría de las filas.

El optimizador de SQL Server decide si puede usar un índice filtrado según el predicado de la consulta y las estadísticas disponibles. La consulta no necesita reproducir literalmente el texto del filtro, pero su predicado debe ser compatible con el subconjunto de filas cubierto por el índice. El filtro configurado en EF Core debe ser SQL válido para el proveedor.

### Columnas incluidas

Las columnas incluidas son columnas que se añaden al índice no agrupado pero no forman parte de la clave del índice. Se usan para cubrir consultas que necesitan columnas adicionales sin tener que consultar la tabla. Se configuran con IncludeProperties.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasIndex(o => o.Cliente)
        .IncludeProperties(o => new { o.NumeroOrden, o.FechaCreacion, o.Estado })
        .HasDatabaseName("IX_OrdenesFabricacion_Cliente_Incluye");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda crea un índice sobre Cliente. La tercera incluye las columnas NumeroOrden, FechaCreacion y Estado. La cuarta establece el nombre. El índice contiene los valores de Cliente como clave y los valores de las columnas incluidas como datos adicionales. Las consultas que filtran por Cliente y seleccionan NumeroOrden, FechaCreacion y Estado se resuelven completamente con el índice, sin consultar la tabla. Este tipo de índice se conoce como índice cubriente.

Error común: si se incluyen demasiadas columnas, el índice ocupa mucho espacio y se actualiza lentamente. Se deben incluir solo las columnas necesarias para las consultas más frecuentes.

## Restricciones CHECK

Una restricción CHECK es una regla que la base de datos aplica al insertar o actualizar una fila. Se configura con `HasCheckConstraint` sobre la tabla. La restricción se expresa como una condición SQL que debe cumplirse.

```
modelBuilder.Entity<PlanchaAcero>(entity =>
{
    entity.ToTable(t =>
    {
        t.HasCheckConstraint("CK_PlanchasAcero_Espesor", "[Espesor] > 0");
        t.HasCheckConstraint("CK_PlanchasAcero_Peso", "[Peso] > 0");
    });
});
```

CSHARP

La primera línea selecciona la entidad. La segunda configura la tabla. La tercera añade la restricción sobre Espesor. La cuarta añade la restricción sobre Peso. Las restricciones se incluyen en el script de creación de la tabla y se aplican en cada inserción o actualización.

Error común: si la restricción CHECK usa una sintaxis SQL incorrecta, la migración falla al crear la tabla. La expresión debe ser válida para el motor de base de datos.

## Restricciones DEFAULT

Una restricción DEFAULT establece el valor que se asigna a una columna cuando no se especifica en el INSERT. Se configura con `HasDefaultValue` o `HasDefaultValueSql`.

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.Property(o => o.Estado)
        .HasDefaultValue("Pendiente");

    entity.Property(o => o.FechaCreacion)
        .HasDefaultValueSql("GETDATE()");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda selecciona la propiedad Estado. La tercera establece el valor por defecto "Pendiente". La cuarta selecciona la propiedad FechaCreacion. La quinta establece la expresión SQL `GETDATE()` como valor por defecto.

Error común: esperar que el valor por defecto de la base de datos sustituya cualquier valor enviado por la aplicación. Tanto `HasDefaultValue` como `HasDefaultValueSql` se aplican cuando la columna se omite del INSERT; si EF Core envía explícitamente un valor como `string.Empty`, SQL Server conserva ese valor y no usa el DEFAULT. Si se quiere depender del valor de base de datos, la propiedad y su valor centinela deben configurarse de modo que EF pueda omitir la columna, o bien debe asignarse el valor de dominio explícitamente en la aplicación.

## Restricciones UNIQUE

Una restricción UNIQUE garantiza que no haya dos filas con el mismo valor en las columnas especificadas. Se configura con `HasAlternateKey` o con `HasIndex().IsUnique()`.

```
modelBuilder.Entity<Aleacion>(entity =>
{
    entity.HasAlternateKey(a => a.Codigo)
        .HasName("AK_Aleaciones_Codigo");
});
```

CSHARP

La primera línea selecciona la entidad. La segunda declara Codigo como clave alternativa. La tercera establece el nombre. La restricción UNIQUE se crea en la base de datos.

La diferencia entre `HasAlternateKey` y `HasIndex().IsUnique()` es que la clave alternativa puede ser referenciada por una clave foránea, mientras que el índice único no.

Error común: si se declara Codigo como clave alternativa y también se crea un índice único con `HasIndex().IsUnique()`, EF Core crea dos índices únicos sobre la misma columna. Se debe usar solo una de las dos formas.

## Índice agrupado vs no agrupado con el proveedor de SQL Server

En SQL Server sólo puede existir un índice agrupado por tabla. Por defecto, la clave primaria queda respaldada por un índice agrupado y los demás índices son no agrupados. El proveedor de EF Core para SQL Server sí permite configurar el clustering mediante Fluent API con `IsClustered()`.

Si se quiere agrupar físicamente la tabla por FechaCreacion en lugar de por la clave primaria, hay que marcar explícitamente la clave como no agrupada antes de marcar el índice de FechaCreacion como agrupado:

```
modelBuilder.Entity<OrdenFabricacion>(entity =>
{
    entity.HasKey(o => o.Id)
        .IsClustered(false);

    entity.HasIndex(o => o.FechaCreacion)
        .IsClustered();
});
```

CSHARP

La primera configuración mantiene Id como clave primaria pero hace no agrupado su índice. La segunda crea el índice de FechaCreacion como agrupado. Ambas decisiones son específicas del proveedor de SQL Server.

Error común: marcar un segundo índice como agrupado sin desagrupar antes la clave primaria. SQL Server admite un único índice agrupado por tabla, por lo que esa configuración entra en conflicto con el clustering predeterminado de la clave primaria.

El impacto de los índices en el rendimiento

Los índices aceleran las consultas de lectura pero ralentizan las operaciones de escritura. Cada vez que se inserta, actualiza o elimina una fila, la base de datos debe actualizar todos los índices afectados. Por eso, no se deben crear índices indiscriminadamente. Se deben crear solo los índices que se usan en las consultas frecuentes.

```
// Bueno: índice sobre una columna usada en filtros frecuentes
entity.HasIndex(o => o.Cliente);

// Malo: índice sobre una columna que nunca se usa en filtros
entity.HasIndex(o => o.Observaciones);
```

CSHARP

La primera línea crea un índice sobre Cliente, que se usa frecuentemente en filtros. La segunda crea un índice sobre Observaciones, que rara vez se usa en filtros. El segundo índice ocupa espacio y ralentiza las escrituras sin aportar beneficio.

Error común: si se crean demasiados índices, las operaciones de escritura se vuelven lentas y el tamaño de la base de datos crece innecesariamente. Se debe revisar periódicamente qué índices se usan y eliminar los que no aportan beneficio.

## El proyecto AceriaData

En el proyecto AceriaData, se configuran en este punto los índices y las restricciones de las entidades. Se crean índices únicos sobre los identificadores naturales, índices compuestos para las consultas frecuentes, índices filtrados para las consultas por estado, columnas incluidas para las consultas cubrientes, restricciones CHECK para las validaciones de negocio y restricciones DEFAULT para los valores por defecto. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

## Resumen de la teoría

Un índice acelera las búsquedas pero ralentiza las escrituras.

El índice agrupado determina el orden físico de las filas.

El índice no agrupado es una estructura separada.

El índice único garantiza la unicidad.

El índice compuesto acelera las consultas por varias columnas.

El índice filtrado incluye solo un subconjunto de filas.

Las columnas incluidas permiten crear índices cubrientes.

Las restricciones CHECK validan condiciones de negocio.

Las restricciones DEFAULT establecen valores por defecto.

Las restricciones UNIQUE garantizan la unicidad.

EF Core configura los índices con HasIndex.

Las restricciones CHECK se configuran con HasCheckConstraint.

Las restricciones DEFAULT se configuran con HasDefaultValue o HasDefaultValueSql.

Las restricciones UNIQUE se configuran con HasAlternateKey o HasIndex().IsUnique().

## Punto 2.10 - Filtros globales de consulta

**Audiencia:** Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

**Proyecto:** Se incorporan filtros globales al modelo acumulativo de AceriaData mediante HasQueryFilter y se aprende a omitirlos de forma explícita con IgnoreQueryFilters.

### Objetivos de aprendizaje

Comprender qué es un filtro global de consulta.

Configurar HasQueryFilter.

Combinar condiciones de filtrado.

Usar IgnoreQueryFilters de forma controlada.

Comprender su efecto en consultas y navegaciones.

Preparar el modelo para Soft Delete sin introducir todavía IsDeleted.

### Teoría

#### Qué es un filtro global de consulta

Un filtro global de consulta es una condición que EF Core aplica automáticamente a todas las consultas que involucran a una entidad. Se configura en el método OnModelCreating con el método HasQueryFilter. Una vez configurado, cualquier consulta LINQ sobre esa entidad incluye la condición del filtro sin que el programador tenga que escribirla explícitamente.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasQueryFilter(o => o.Estado != "Cancelada");
```

CSHARP

La primera línea selecciona la entidad OrdenFabricacion. La segunda configura un filtro global que excluye las órdenes cuyo estado es "Cancelada". A partir de este momento, cualquier consulta sobre OrdenesFabricacion devuelve solo las órdenes que no están canceladas. Si se quiere incluir las canceladas, se debe usar IgnoreQueryFilters.

Los filtros globales son útiles para aplicar reglas de negocio de forma transversal. Por ejemplo, en una aplicación multi-tenant, se puede filtrar por el identificador del tenant. En una aplicación con Soft Delete, se puede filtrar por la propiedad IsDeleted. En una aplicación con datos históricos, se puede filtrar por la fecha de vigencia.

#### Configurar un filtro global

El filtro global se configura con HasQueryFilter sobre la entidad. La condición se expresa como una expresión lambda que devuelve un booleano. EF Core traduce esa expresión a una cláusula WHERE en todas las consultas.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasQueryFilter(o => o.Estado != "Cancelada");

modelBuilder.Entity<PlanchaAcero>()
    .HasQueryFilter(p => p.Activa);

modelBuilder.Entity<Aleacion>()
    .HasQueryFilter(a => a.Activo);
```

CSHARP

La primera línea configura el filtro para OrdenFabricacion, excluyendo las canceladas. La segunda línea configura el filtro para PlanchaAcero, excluyendo las inactivas. La tercera línea configura el filtro para Aleacion, excluyendo las inactivas. Cada filtro se aplica solo a la entidad correspondiente.

Los filtros globales también interactúan con las relaciones. Las entidades relacionadas que no cumplen su propio filtro pueden quedar fuera del resultado y, cuando intervienen navegaciones requeridas, el SQL generado puede usar INNER JOIN y hacer que desaparezcan también filas del otro extremo. Por eso los filtros deben diseñarse junto con la cardinalidad y obligatoriedad de las relaciones.

#### Combinar varios filtros

Se pueden combinar varios filtros globales sobre la misma entidad usando operadores lógicos. La condición del filtro puede ser tan compleja como se necesite.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasQueryFilter(o => o.Estado != "Cancelada" && o.FechaCreacion.Year >= 2020);
```

CSHARP

La primera línea configura un filtro que excluye las órdenes canceladas y las anteriores al año 2020. La condición se traduce a una cláusula WHERE con dos condiciones unidas por AND.

Error común: si el filtro global es demasiado restrictivo, puede excluir datos que se necesitan en algunas consultas. En ese caso, se debe usar IgnoreQueryFilters para omitir el filtro en consultas concretas.

#### Ignorar filtros globales

El método IgnoreQueryFilters permite omitir los filtros globales en una consulta concreta. Se aplica sobre el IQueryable antes de materializar la consulta.

```
var todasLasOrdenes = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .ToList();
```

CSHARP

La primera línea inicia la consulta sobre OrdenesFabricacion. La segunda omite los filtros globales. La tercera materializa la consulta. El resultado incluye todas las órdenes, incluso las que cumplen las condiciones de los filtros.

Error común: si se usa IgnoreQueryFilters sin necesidad, se pueden cargar datos que no deberían estar visibles para el usuario. Se debe usar solo cuando sea estrictamente necesario y documentar la razón.

## El proyecto AceriaData

En 2.10 el estado ejecutable aplica filtros de negocio ya disponibles: las órdenes canceladas no aparecen en la consulta normal, las planchas inactivas quedan excluidas y las relaciones OrdenAleacion no activas se ocultan. No se añaden todavía IsDeleted ni DeletedAt.

## Resumen de la teoría

HasQueryFilter agrega una condición transversal a las consultas de una entidad.

IgnoreQueryFilters permite desactivar expresamente los filtros para una consulta concreta.

Un filtro no borra filas de la base de datos.

Los filtros pueden afectar a las entidades relacionadas.

Soft Delete se implementa en el punto 2.11.

## Punto 2.11 – Soft Delete: implementación, consultas y restauración

Audiencia: Desarrolladores con conocimientos básicos de programación y SQL, sin experiencia previa en ORMs ni en Entity Framework Core.

Proyecto: Se implementa el patrón Soft Delete sobre el modelo acumulativo de AceriaData con IsDeleted, DeletedAt, HasQueryFilter e IgnoreQueryFilters.

## Objetivos de aprendizaje

Comprender el borrado lógico.

Añadir IsDeleted y DeletedAt.

Combinar filtros de negocio y Soft Delete.

Restaurar registros eliminados lógicamente.  
 Distinguir borrado lógico de borrado físico.  
 Analizar el efecto de los filtros sobre relaciones requeridas.

## Teoría

### Qué es Soft Delete

Soft Delete es un patrón que consiste en marcar una entidad como eliminada sin borrarla físicamente de la base de datos. En lugar de ejecutar un DELETE, se actualiza una propiedad como IsDeleted a true. La entidad sigue existiendo en la base de datos, pero se excluye de las consultas mediante un filtro global. Este patrón es útil cuando se quiere conservar el historial de datos, cuando hay relaciones que impiden el borrado físico o cuando se quiere permitir la restauración.

```
public class OrdenFabricacion
{
    public int Id { get; set; }
    public string NumeroOrden { get; set; } = string.Empty;
    public bool IsDeleted { get; set; }
    public DateTime? DeletedAt { get; set; }
}
```

CSHARP

La primera línea declara la propiedad IsDeleted de tipo bool. La segunda línea declara la propiedad DeletedAt de tipo DateTime?. Estas dos propiedades son las que permiten implementar el Soft Delete. IsDeleted indica si la entidad está eliminada. DeletedAt almacena la fecha en la que se eliminó.

### Configurar el filtro global de Soft Delete

El filtro global de Soft Delete se configura con HasQueryFilter sobre la entidad, excluyendo las entidades marcadas como eliminadas.

```
modelBuilder.Entity<OrdenFabricacion>()
    .HasQueryFilter(o => !o.IsDeleted);
```

CSHARP

La primera línea selecciona la entidad. La segunda configura el filtro para excluir las órdenes marcadas como eliminadas. A partir de este momento, cualquier consulta sobre OrdenesFabricacion devuelve solo las órdenes no eliminadas.

Error común: si se olvida configurar el filtro global, las entidades eliminadas siguen apareciendo en las consultas. Se debe configurar el filtro en todas las entidades que implementan Soft Delete.

#### Eliminar con Soft Delete

Para eliminar una entidad con Soft Delete, no se llama a Remove. En su lugar, se marca la propiedad IsDeleted a true y se llama a SaveChanges.

```
var orden = context.OrdenesFabricacion.FirstOrDefault(o => o.Id == 1);
orden!.IsDeleted = true;
orden.DeletedAt = DateTime.Now;
context.SaveChanges();
```

CSHARP

La primera línea carga la orden. La segunda marca IsDeleted a true. La tercera asigna la fecha de eliminación. La cuarta ejecuta el UPDATE que marca la orden como eliminada. La entidad sigue existiendo en la base de datos, pero ya no aparece en las consultas.

Error común: si se usa Remove en lugar de marcar IsDeleted, la entidad se elimina físicamente y se pierde el historial. Se debe usar el patrón Soft Delete de forma consistente.

#### Restaurar una entidad eliminada

Para restaurar una entidad eliminada, se carga con IgnoreQueryFilters, se marca IsDeleted a false y se llama a SaveChanges.

```
var orden = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .FirstOrDefault(o => o.Id == 1);

orden!.IsDeleted = false;
```

CSHARP

```
orden.DeletedAt = null;
context.SaveChanges();
```

La primera línea inicia la consulta. La segunda omite los filtros globales para poder cargar la entidad eliminada. La tercera busca la orden por Id. La cuarta marca IsDeleted a false. La quinta borra la fecha de eliminación. La sexta ejecuta el UPDATE que restaura la orden.

Error común: si se intenta cargar una entidad eliminada sin IgnoreQueryFilters, la consulta devuelve null porque el filtro global la excluye. Se debe usar IgnoreQueryFilters para cargarla.

Eliminar físicamente una entidad con Soft Delete

Para eliminar físicamente una entidad que implementa Soft Delete, se carga con IgnoreQueryFilters, se llama a Remove y se llama a SaveChanges.

```
var orden = context.OrdenesFabricacion
    .IgnoreQueryFilters()
    .FirstOrDefault(o => o.Id == 1);

context.OrdenesFabricacion.Remove(orden!);
context.SaveChanges();
```

CSHARP

La primera línea inicia la consulta. La segunda omite los filtros globales. La tercera busca la orden por Id. La cuarta marca la orden para eliminar. La quinta ejecuta el DELETE que borra la entidad físicamente.

Para borrar físicamente un registro previamente ocultado por Soft Delete, una consulta normal no podrá recuperarlo porque HasQueryFilter lo excluye. Por eso se usa IgnoreQueryFilters para localizarlo antes de llamar a Remove. Remove no realiza por sí mismo una búsqueda en la base de datos: actúa sobre la instancia que se le entrega.

Implicaciones en las relaciones

Los filtros globales se aplican también a las consultas que cargan entidades relacionadas. Si una entidad principal tiene una colección de entidades dependientes y el filtro global de las dependientes excluye algunas, la colección solo incluye las que cumplen el filtro.

```
var orden = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .FirstOrDefault(o => o.Id == 1);
```

CSHARP

La primera línea inicia la consulta. La segunda incluye la colección de planchas. La tercera busca la orden por Id. Si el filtro global de PlanchaAcero excluye las planchas inactivas, la colección Planchas solo incluye las planchas activas. Las planchas inactivas no se cargan.

Error común: si se espera que la colección incluya todas las entidades relacionadas, pero el filtro global excluye algunas, el resultado puede ser confuso. Se debe usar IgnoreQueryFilters en la consulta si se quieren incluir todas.

## Filtros globales y navegaciones requeridas

Los filtros globales también afectan al SQL que EF Core genera al cargar relaciones. Hay que prestar especial atención a las navegaciones requeridas: EF Core puede utilizar un INNER JOIN y, si la entidad relacionada queda excluida por su filtro global, la fila de la entidad que se está consultando también puede desaparecer del resultado.

```
var planchas = context.PlanchasAcero
    .Include(p => p.Orden)
    .ToList();
```

CSHARP

Si `PlanchaAcero.Orden` es requerida y `OrdenFabricacion` tiene un filtro que excluye una orden, la consulta con `Include` puede devolver menos planchas de las esperadas porque el INNER JOIN elimina las filas cuya orden relacionada fue filtrada. No debe asumirse simplemente que la navegación requerida quedará a `null`.

Una forma de mantener resultados coherentes es configurar filtros compatibles en ambos extremos de la relación cuando ambos tipos participan en la misma regla de visibilidad. Otra posibilidad, cuando el dominio realmente lo permite, es hacer opcional la navegación para que el SQL pueda usar un LEFT JOIN. `IgnoreQueryFilters()` debe utilizarse sólo en consultas donde se necesite omitir deliberadamente esas reglas.

## El proyecto AceriaData

En el proyecto AceriaData, se implementa el Soft Delete en las entidades OrdenFabricacion, PlanchaAcero, Aleacion y EstadoOrden. Se añaden las propiedades IsDeleted y DeletedAt a cada entidad, se configura el filtro global con HasQueryFilter y se añaden métodos para eliminar, restaurar y eliminar físicamente. La base de datos sigue siendo AceriaDB en SQL Server LocalDB.

## Resumen de la teoría

Soft Delete conserva físicamente el registro y modifica su estado lógico.

IsDeleted indica si la entidad está eliminada y DeletedAt registra el momento del borrado lógico.

HasQueryFilter oculta automáticamente los registros eliminados en las consultas normales.

IgnoreQueryFilters permite omitir deliberadamente el filtro en tareas administrativas como restauración o auditoría.

Para eliminar lógicamente una entidad se marca IsDeleted a true, se establece DeletedAt y se guardan los cambios; no se usa Remove.

Para restaurarla se carga con IgnoreQueryFilters, se marca IsDeleted a false, se limpia DeletedAt y se guardan los cambios.

Remove continúa representando un borrado físico y no debe confundirse con Soft Delete.

Los filtros globales también afectan a las entidades relacionadas. En navegaciones requeridas, un INNER JOIN puede eliminar filas del resultado si la entidad relacionada queda excluida por su filtro.

Cuando ambos extremos de una relación participan en la misma regla de visibilidad, deben configurarse filtros compatibles; si el dominio lo permite, una navegación opcional puede evitar el efecto de un INNER JOIN.

En AceriaData, Soft Delete se aplica a las entidades principales manteniendo SQL Server LocalDB y el historial mediante Migrations.

## Punto 2.12 - Integración de EF Core en Clean Architecture y Arquitectura Hexagonal

Audiencia: Desarrolladores que ya han completado el modelado acumulativo del módulo.

Proyecto: AceriaData se reorganiza en Domain, Application, Infraestructure y Console, manteniendo el mismo dominio y aislando EF Core en infraestructura.

## Objetivos de aprendizaje

Separar responsabilidades por capas.

Mantener Domain y Application independientes de EF Core.

Usar repositorios y unidad de trabajo como puertos de aplicación.

Implementar esos puertos en Infraestructure.

Registrar la infraestructura mediante inyección de dependencias.

Conservar migraciones y comportamiento durante el refactor.

## Teoría

### Repositorio y Unidad de Trabajo como puertos de aplicación

#### Alcance: Repositorio y Unidad de Trabajo

Comprender qué es el patrón Repositorio y qué problema resuelve.

Diferenciar entre repositorio genérico y repositorio específico.

Comprender qué es el patrón Unidad de Trabajo.

Diseñar interfaces de repositorio que abstraigan EF Core.

Implementar repositorios con EF Core.

Implementar una unidad de trabajo que coordine varios repositorios.

Registrar los repositorios y la unidad de trabajo en el contenedor de dependencias.

Refactorizar el proyecto AceriaData para usar repositorios.

Identificar anti-patrones en el uso del patrón Repositorio.

## Qué es el patrón Repositorio

El patrón Repositorio es un patrón de diseño que encapsula la lógica de acceso a datos en una clase intermedia entre la capa de negocio y la capa de persistencia. Su objetivo es presentar una interfaz que simule una colección en memoria de objetos del dominio, ocultando los detalles de la base de datos. La capa de negocio no sabe si los datos vienen de SQL Server, de SQLite, de un servicio web o de una lista en memoria. Solo conoce la interfaz del repositorio.

```
public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

CSHARP

La primera línea declara la interfaz IOrdenRepositorio. La segunda declara el método que obtiene una orden por Id. La tercera declara el método que obtiene todas las órdenes. La cuarta declara el método que agrega una orden. La quinta declara el método que elimina una orden. La interfaz no menciona EF Core, ni DbContext, ni DbSet. Solo menciona entidades del dominio.

La implementación del repositorio sí conoce EF Core, pero la capa de negocio no. Esto permite cambiar la implementación sin afectar a la capa de negocio.

```
public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context)
    {
        _context = context;
    }

    public OrdenFabricacion? ObtenerPorId(int id)
    {
        return _context.OrdenesFabricacion.FirstOrDefault(o => o.Id == id);
    }

    public List<OrdenFabricacion> ObtenerTodas()
    {
        return _context.OrdenesFabricacion.ToList();
    }

    public void Agregar(OrdenFabricacion orden)
    {
        _context.OrdenesFabricacion.Add(orden);
    }

    public void Eliminar(OrdenFabricacion orden)
    {
        _context.OrdenesFabricacion.Remove(orden);
    }
}
```

CSHARP

La primera línea declara la clase que implementa la interfaz. La segunda declara el campo del DbContext. La tercera declara el constructor que recibe el DbContext. La cuarta asigna el parámetro al campo. Las siguientes líneas implementan los métodos de la interfaz usando EF Core.

Por qué usar el patrón Repositorio

El patrón Repositorio aporta varias ventajas. La primera es el desacoplamiento: la capa de negocio no depende de EF Core ni de la base de datos. La segunda es la testabilidad: se puede sustituir el repositorio real por un mock en las pruebas unitarias. La tercera es la centralización: la lógica de acceso a datos está en un solo lugar. La cuarta es la claridad: la interfaz del repositorio expresa las operaciones del dominio en lugar de las operaciones de EF Core.

Sin embargo, el patrón Repositorio también tiene críticas. Algunos desarrolladores consideran que EF Core ya implementa el patrón Repositorio a través del DbSet y que añadir una capa adicional es redundante. Otros consideran que el repositorio genérico es un anti-patrón porque intenta abstraer operaciones que no se pueden abstraer. La decisión de usar repositorios depende del proyecto y del equipo.

## Repositorio genérico

El repositorio genérico es una implementación que funciona para cualquier entidad. Se define una interfaz `IRepositorio<T>` y una implementación `Repositorio<T>`.

```
public interface IRepositorio<T> where T : class
{
    T? ObtenerPorId(int id);
    List<T> ObtenerTodas();
    void Agregar(T entidad);
    void Eliminar(T entidad);
}

public class Repositorio<T> : IRepositorio<T> where T : class
{
    protected readonly AceriaDbContext _context;
    private readonly DbSet<T> _dbSet;

    public Repositorio(AceriaDbContext context)
    {
        _context = context;
        _dbSet = context.Set<T>();
    }

    public T? ObtenerPorId(int id) => _dbSet.Find(id);

    public List<T> ObtenerTodas() => _dbSet.ToList();

    public void Agregar(T entidad) => _dbSet.Add(entidad);

    public void Eliminar(T entidad) => _dbSet.Remove(entidad);
}
```

CSHARP

La primera línea declara la interfaz genérica. La segunda línea declara el método que obtiene una entidad por Id. La tercera línea declara el método que obtiene todas las entidades. La cuarta línea declara el método que agrega una entidad. La quinta línea declara el método que elimina una entidad. La sexta línea declara la implementación genérica. La séptima línea declara el campo del DbContext. La octava línea declara el campo del DbSet. La novena línea declara el constructor.

El repositorio genérico tiene la ventaja de que se escribe una sola vez y sirve para todas las entidades. Pero tiene la desventaja de que no puede expresar operaciones específicas de cada entidad. Por ejemplo, un método `ObtenerPorNumeroOrden` no tiene sentido en el repositorio genérico porque no todas las entidades tienen un número de orden.

## Repositorio específico y trazabilidad con AceriaData

Un repositorio específico expresa operaciones del dominio sin obligar a Application a conocer DbContext, DbSet o LINQ de EF Core. En el estado real 2.12, AceriaData no hereda de un repositorio genérico: define directamente `IOrdenRepositorio` en Application.

```
public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    OrdenFabricacion? ObtenerPorNumero(string numeroOrden);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

CSHARP

Infrastructure implementa exactamente ese puerto:

```
public sealed class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;

    public OrdenRepositorio(AceriaDbContext context) => _context = context;

    public OrdenFabricacion? ObtenerPorId(int id)
    => _context.OrdenesFabricacion.Find(id);

    public OrdenFabricacion? ObtenerPorNumero(string numeroOrden)
    => _context.OrdenesFabricacion.FirstOrDefault(x => x.NumeroOrden == numeroOrden);

    public List<OrdenFabricacion> ObtenerTodas()
    => _context.OrdenesFabricacion.OrderBy(x => x.Id).ToList();
}
```

CSHARP

```

public void Agregar(OrdenFabricacion orden)
    => _context.OrdenesFabricacion.Add(orden);

public void Eliminar(OrdenFabricacion orden)
    => _context.OrdenesFabricacion.Remove(orden);
}

```

La interfaz vive en Application y la implementación vive en Infrastructure. Esta separación es la que utiliza realmente M02/PROYECTO/2.12.

## Unidad de Trabajo

El patrón Unidad de Trabajo concentra el momento en que se confirman los cambios de un caso de uso. El diseño puede coordinar varios repositorios; en el estado 2.12 de AceriaData se expone únicamente el repositorio de órdenes, que es el que necesita el caso de uso implementado.

```

public interface IUnidadDeTrabajo : IDisposable
{
    IOrdenRepositorio Ordenes { get; }
    int Guardar();
}

```

CSHARP

La implementación real de Infrastructure comparte el mismo AceriaDbContext con el repositorio de órdenes y delega la confirmación en SaveChanges:

```

public sealed class UnidadDeTrabajo : IUnidadDeTrabajo
{
    private readonly AceriaDbContext _context;
    private IOrdenRepositorio? _ordenes;

    public UnidadDeTrabajo(AceriaDbContext context) => _context = context;

    public IOrdenRepositorio Ordenes
        => _ordenes ??= new OrdenRepositorio(_context);

    public int Guardar() => _context.SaveChanges();

    public void Dispose() => _context.Dispose();
}

```

CSHARP

El caso de uso de Application trabaja sólo contra IUnidadDeTrabajo. Añade la entidad mediante el puerto y confirma una vez al final:

```

public int Ejecutar(string numeroOrden, string cliente)
{
    _unidad.Ordenes.Agregar(new OrdenFabricacion
    {
        NumeroOrden = numeroOrden,
        Cliente = cliente,
        FechaCreacion = DateTime.UtcNow,
        Estado = "Pendiente"
    });

    return _unidad.Guardar();
}

```

CSHARP

Así, Application no conoce EF Core. Si el sistema incorpora en el futuro repositorios de planchas o aleaciones, pueden añadirse nuevos puertos a Application y coordinarlos desde la misma unidad de trabajo sin cambiar esta regla de dependencias.

## Registro en el contenedor de dependencias

En AceriaData, Infrastructure expone una extensión que registra SQL Server, el repositorio específico y la unidad de trabajo con ciclo de vida Scoped.

```

public static IServiceCollection AddAceriaInfrastructure(
    this IServiceCollection services,
    string connectionString)
{
    services.AddDbContext<AceriaDbContext>(
        o => o.UseSqlServer(connectionString));
}

```

CSHARP

```

services.AddScoped<IOrdenRepositorio, OrdenRepositorio>();
services.AddScoped<IUnidadDeTrabajo, UnidadDeTrabajo>();

return services;
}

```

El DbContext, OrdenRepositorio y UnidadDeTrabajo se resuelven dentro del mismo ámbito. La capa Console actúa como composition root: llama a AddAceriaInfrastructure y registra el caso de uso, mientras Domain y Application permanecen libres de referencias a EF Core.

#### Anti-patrones del patrón Repositorio

El patrón Repositorio tiene varios anti-patrones que se deben evitar. El primero es el repositorio que expone IQueryable<T>. Esto rompe la abstracción porque la capa de negocio puede componer consultas que el repositorio no controla.

```

// Anti-patrón
public interface IOrdenRepositorio
{
    IQueryable<OrdenFabricacion> ObtenerTodas();
}

```

CSHARP

La primera línea declara la interfaz. La segunda línea expone IQueryable<OrdenFabricacion>. La capa de negocio puede añadir filtros, ordenaciones y proyecciones sin que el repositorio lo sepa. Esto rompe el encapsulamiento y puede provocar consultas ineficientes.

El segundo anti-patrón es el repositorio que expone operaciones de EF Core como Include, AsNoTracking o FromSql. Esto acopla la capa de negocio a EF Core.

```

// Anti-patrón
public interface IOrdenRepositorio
{
    List<OrdenFabricacion> ObtenerTodasConPlanchas();
    List<OrdenFabricacion> ObtenerTodasSinTracking();
}

```

CSHARP

La primera línea declara la interfaz. La segunda línea expone un método que usa Include. La tercera línea expone un método que usa AsNoTracking. Los nombres de los métodos revelan detalles de EF Core que la capa de negocio no debería conocer.

El tercer anti-patrón es el repositorio genérico que intenta cubrir todas las operaciones posibles. Cuanto más genérico es el repositorio, más difícil es de usar y más propenso a errores.

```

// Anti-patrón
public interface IRepository<T>
{
    T? ObtenerPorId(int id);
    List<T> ObtenerTodas();
    List<T> ObtenerPorExpresion(Expression<Func<T, bool>> predicado);
    List<T> ObtenerConInclude(params Expression<Func<T, object>>[] includes);
    void Agregar(T entidad);
    void Actualizar(T entidad);
    void Eliminar(T entidad);
    void EliminarPorExpresion(Expression<Func<T, bool>> predicado);
}

```

CSHARP

La primera línea declara la interfaz genérica. Las siguientes líneas exponen métodos que revelan detalles de EF Core. El repositorio se convierte en una envoltura de DbSet sin aportar valor.

#### Cuándo usar el patrón Repositorio

El patrón Repositorio es adecuado cuando el proyecto tiene una capa de dominio bien definida, cuando se quiere desacoplar la capa de negocio de EF Core, cuando se quiere facilitar el testing unitario y cuando se trabaja con múltiples fuentes de datos. No es adecuado cuando el proyecto es pequeño, cuando el equipo conoce bien EF Core y cuando se quiere aprovechar toda la potencia de EF Core sin abstracciones adicionales.

En el proyecto AceriaData, el patrón Repositorio se usa para encapsular el acceso a las entidades principales y para preparar la arquitectura limpia del punto 2.12. La unidad de trabajo coordina los repositorios y garantiza la coherencia transaccional.

## Síntesis: Repositorio y Unidad de Trabajo

El patrón Repositorio encapsula el acceso a datos en una clase intermedia.

La capa de negocio no depende de EF Core ni de la base de datos.

Un repositorio genérico puede reutilizar operaciones comunes, pero no siempre aporta valor y no es la opción utilizada por AceriaData 2.12.

El repositorio específico expresa operaciones propias del caso de uso y mantiene EF Core fuera de Application.

El patrón Unidad de Trabajo coordina varios repositorios bajo una misma transacción.

El repositorio no llama a SaveChanges: eso lo hace la unidad de trabajo.

Los repositorios y la unidad de trabajo se registran con ciclo de vida Scoped.

Los anti-patrones son: exponer IQueryable, exponer operaciones de EF Core y usar repositorios genéricos excesivamente amplios.

En el proyecto AceriaData 2.12 se implementan un repositorio específico de órdenes y una unidad de trabajo, ambos como adaptadores/puertos trazables con el código final.

## Clean Architecture y Arquitectura Hexagonal

### Alcance: Clean Architecture y Arquitectura Hexagonal

Comprender los principios de la arquitectura limpia.

Comprender los principios de la Arquitectura Hexagonal (puertos y adaptadores).

Identificar las capas de una aplicación: dominio, aplicación, infraestructura y presentación.

Aplicar el principio de inversión de dependencias.

Separar el proyecto AceriaData en proyectos por capa.

Colocar las entidades en Domain y los puertos de persistencia (como IOrdenRepositorio e IUnidadDeTrabajo) en Application.

Colocar la implementación de EF Core en la capa de infraestructura.

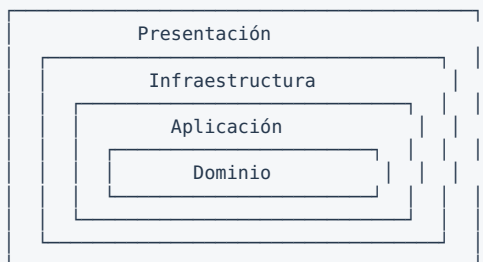
Colocar los casos de uso en la capa de aplicación.

Comprender la regla de dependencia: las capas internas no conocen las externas.

### Qué es la arquitectura limpia

La arquitectura limpia es un conjunto de principios de diseño que organizan el código en capas concéntricas, donde las capas internas no conocen las capas externas. El objetivo es que la lógica de negocio sea independiente de los detalles de infraestructura, como la base de datos, la interfaz de usuario o los servicios externos. La regla fundamental es la regla de dependencia: las dependencias apuntan hacia dentro. Domain no depende de Infrastructure. Application depende de Domain y define los puertos que necesita. Infrastructure depende de Application y Domain para implementar esos puertos. Los adaptadores de entrada invocan Application; en AceriaData, Console actúa además como composition root y por ello conoce Infrastructure únicamente para registrar y componer las implementaciones concretas.

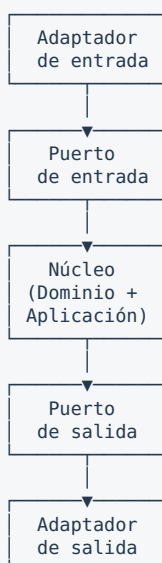
TEXT



El diagrama representa la dirección conceptual de las dependencias: Domain queda en el centro; Application depende de Domain; Infrastructure depende de Application y Domain para implementar los puertos; y los adaptadores de entrada invocan Application. En AceriaData, Console es además el composition root y referencia Infrastructure únicamente para registrar y resolver los adaptadores concretos.

## Qué es la Arquitectura Hexagonal

La Arquitectura Hexagonal, también conocida como arquitectura de puertos y adaptadores, es una variante de la arquitectura limpia que enfatiza la separación entre el núcleo de la aplicación y los adaptadores externos. El núcleo contiene la lógica de negocio y define los puertos, que son interfaces que expresan lo que el núcleo necesita del exterior. Los adaptadores son las implementaciones concretas de esos puertos: adaptadores de entrada (controladores, endpoints) y adaptadores de salida (repositorios, servicios externos).



TEXT

El diagrama muestra el flujo. El adaptador de entrada recibe la petición y la traduce a una llamada al puerto de entrada. El puerto de entrada es una interfaz que el núcleo implementa. El núcleo ejecuta la lógica de negocio y llama al puerto de salida. El puerto de salida es una interfaz que el adaptador de salida implementa. El adaptador de salida se comunica con el exterior.

La regla de dependencia

La regla de dependencia establece que las dependencias del código fuente deben apuntar hacia dentro. Domain no conoce Application ni Infrastructure. Application conoce Domain, pero no Infrastructure. Infrastructure conoce Application y Domain porque implementa sus puertos. El código de entrada usa Application; el composition root puede referenciar Infrastructure para registrar los adaptadores sin trasladar esa dependencia a los casos de uso.

```

// Dominio: no conoce nada
namespace AceriaData.Domain
{
    public class OrdenFabricacion
    {
        public int Id { get; set; }
        public string NumeroOrden { get; set; } = string.Empty;
    }
}

// Aplicación: conoce el dominio
namespace AceriaData.Application
{
    public interface IOordenRepositorio
    {
        OrdenFabricacion? ObtenerPorId(int id);
    }
}

// Infraestructura: conoce la aplicación y el dominio
namespace AceriaData.Infrastructure
{
    public class OrdenRepositorio : IOordenRepositorio
    {
        private readonly AceriaDbContext _context;
        public OrdenRepositorio(AceriaDbContext context) { _context = context; }
        public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    }
}

```

CSHARP

La primera sección declara la entidad en el dominio. La segunda sección declara la interfaz del repositorio en la aplicación. La tercera sección declara la implementación del repositorio en la infraestructura. La infraestructura conoce la aplicación porque implementa su interfaz. La aplicación no conoce la infraestructura porque solo depende de la interfaz. La inversión de dependencias se aplica en la dirección de la implementación.

Las capas de la aplicación

La arquitectura limpia organiza el código en capas con responsabilidades separadas. Domain contiene las entidades, los objetos de valor y las reglas de negocio. Application contiene los casos de uso, los DTOs y los puertos que necesita el núcleo, incluidos `IOrdenRepositorio` e `IUnidadDeTrabajo`. Infraestructura contiene el `DbContext`, las migraciones y los adaptadores que implementan esos puertos con EF Core u otras tecnologías. La capa de entrada contiene controladores, endpoints, vistas o la interfaz de consola; en `AceriaData`, `Console` también es el composition root que registra las dependencias concretas.

```
// Capa de dominio
namespace AceriaData.Domain.Entities
{
    public class OrdenFabricacion
    {
        public int Id { get; set; }
        public string NumeroOrden { get; set; } = string.Empty;
        public string Cliente { get; set; } = string.Empty;
        public string Estado { get; set; } = string.Empty;
    }
}

// Capa de aplicación
namespace AceriaData.Application.Interfaces
{
    public interface IOrdenRepositorio
    {
        OrdenFabricacion? ObtenerPorId(int id);
        List<OrdenFabricacion> ObtenerTodas();
        void Agregar(OrdenFabricacion orden);
    }
}

// Capa de infraestructura
namespace AceriaData.Infraestructure.Persistence
{
    public class OrdenRepositorio : IOrdenRepositorio
    {
        private readonly AceriaDbContext _context;
        public OrdenRepositorio(AceriaDbContext context) { _context = context; }
        public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
        public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.ToList();
        public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
    }
}
```

CSHARP

La primera sección declara la entidad en el dominio. La segunda sección declara la interfaz en la aplicación. La tercera sección declara la implementación en la infraestructura. Cada capa tiene su propio espacio de nombres y su propia responsabilidad.

## Inversión de dependencias

La inversión de dependencias es el principio que permite que las capas internas no dependan de las externas. En lugar de que la aplicación dependa de la infraestructura, la infraestructura depende de la aplicación. La aplicación define una interfaz y la infraestructura la implementa. La aplicación no conoce la implementación concreta.

```
// Aplicación define la interfaz
public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
}

// Infraestructura implementa la interfaz
public class OrdenRepositorio : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorio(AceriaDbContext context) { _context = context; }
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
}

// La aplicación usa la interfaz
public class ServicioOrdenes
{
    private readonly IOrdenRepositorio _repositorio;
```

CSHARP

```
public ServicioOrdenes(IOrdenRepositorio repositorio) { _repositorio = repositorio; }
}
```

La primera sección declara la interfaz en la aplicación. La segunda sección declara la implementación en la infraestructura. La tercera sección declara el servicio en la aplicación. El servicio depende de la interfaz, no de la implementación. La infraestructura inyecta la implementación en tiempo de ejecución.

#### El adaptador de salida

El adaptador de salida es la implementación concreta de un puerto de salida. En el proyecto AceriaData, el adaptador de salida es el repositorio que usa EF Core para comunicarse con SQL Server. El puerto de salida es la interfaz del repositorio que define las operaciones que la aplicación necesita.

```
// Puerto de salida (interfaz en la aplicación)
public interface IOrdenRepositorio
{
    OrdenFabricacion? ObtenerPorId(int id);
    List<OrdenFabricacion> ObtenerTodas();
    void Agregar(OrdenFabricacion orden);
}

// Adaptador de salida (implementación en la infraestructura)
public class OrdenRepositorioEfCore : IOrdenRepositorio
{
    private readonly AceriaDbContext _context;
    public OrdenRepositorioEfCore(AceriaDbContext context) { _context = context; }
    public OrdenFabricacion? ObtenerPorId(int id) => _context.OrdenesFabricacion.Find(id);
    public List<OrdenFabricacion> ObtenerTodas() => _context.OrdenesFabricacion.ToList();
    public void Agregar(OrdenFabricacion orden) => _context.OrdenesFabricacion.Add(orden);
}
```

La primera sección declara el puerto. La segunda sección declara el adaptador. El adaptador implementa el puerto usando EF Core. Si se quisiera cambiar a Dapper o a una API externa, se crearía otro adaptador que implemente la misma interfaz.

#### El adaptador de entrada

El adaptador de entrada es el componente que recibe las peticiones del exterior y las traduce a llamadas al núcleo. En una aplicación de consola, el adaptador de entrada es el método Main. En una API REST, el adaptador de entrada son los endpoints. En una aplicación de escritorio, el adaptador de entrada son los botones y los formularios.

```
// Adaptador de entrada (consola)
public class Program
{
    public static void Main()
    {
        var servicio = new ServicioOrdenes(_repositorio);
        var ordenes = servicio.ObtenerTodasLasOrdenes();
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"{orden.NumeroOrden} - {orden.Cliente}");
        }
    }
}
```

La primera línea declara la clase Program. La segunda declara el método Main. La tercera crea el servicio. La cuarta llama al servicio. La quinta itera sobre las órdenes. La sexta muestra los datos. El método Main es el adaptador de entrada: recibe la petición del usuario y la traduce a una llamada al servicio.

#### Los casos de uso

Los casos de uso son las operaciones que la aplicación ofrece al exterior. Se implementan en la capa de aplicación y orquestan las entidades y los repositorios. Cada caso de uso representa una acción concreta del dominio.

```
public class CrearOrdenUseCase
{
    private readonly IOrdenRepositorio _repositorio;

    public CrearOrdenUseCase(IOrdenRepositorio repositorio)
    {
        _repositorio = repositorio;
    }

    public void Ejecutar(string numeroOrden, string cliente)
    {
        var orden = new OrdenFabricacion
        {

```

```

        NumeroOrden = numeroOrden,
        Cliente = cliente,
        Estado = "Pendiente"
    };
    _repositorio.Agregar(orden);
}
}

```

La primera línea declara la clase. La segunda declara el campo del repositorio. La tercera declara el constructor. La cuarta asigna el parámetro al campo. La quinta declara el método Ejecutar. La sexta crea la entidad. La séptima agrega la entidad al repositorio. El caso de uso orquesta la operación sin conocer EF Core.

#### Las entidades de dominio

Las entidades de dominio son clases que representan conceptos del negocio. No tienen dependencias de EF Core ni de ninguna otra tecnología. Son clases POCO (Plain Old CLR Objects) con propiedades y, opcionalmente, métodos que encapsulan reglas de negocio.

```

public class OrdenFabricacion
{
    public int Id { get; private set; }
    public string NumeroOrden { get; private set; }
    public string Cliente { get; private set; }
    public string Estado { get; private set; }

    public OrdenFabricacion(string numeroOrden, string cliente)
    {
        NumeroOrden = numeroOrden;
        Cliente = cliente;
        Estado = "Pendiente";
    }

    public void CambiarEstado(string nuevoEstado)
    {
        if (string.IsNullOrEmpty(nuevoEstado))
        {
            throw new ArgumentException("El estado no puede estar vacío.");
        }
        Estado = nuevoEstado;
    }
}

```

CSHARP

La primera línea declara la clase. La segunda declara la propiedad Id con setter privado. La tercera declara la propiedad NumeroOrden. La cuarta declara la propiedad Cliente. La quinta declara la propiedad Estado. La sexta declara el constructor. La séptima asigna el número de orden. La octava asigna el cliente. La novena asigna el estado inicial. La décima declara el método CambiarEstado. La undécima valida el nuevo estado. La duodécima lanza una excepción si el estado está vacío. La decimotercera asigna el nuevo estado. La entidad encapsula sus reglas de negocio.

#### Configuración de EF Core en la infraestructura

La configuración de EF Core se coloca en Infrastructure. Incluye el DbContext, las configuraciones de entidades con Fluent API y las migraciones. Domain no conoce EF Core. Application usa las entidades de Domain y define los puertos de persistencia que necesita, sin conocer EF Core ni Infrastructure.

```

namespace AceriaData.Infrastructure.Persistence
{
    public class AceriaDbContext : DbContext
    {
        public DbSet<OrdenFabricacion> OrdenesFabricacion { get; set; } = null!;

        public AceriaDbContext(DbContextOptions<AceriaDbContext> options) : base(options) { }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.ApplyConfigurationsFromAssembly(typeof(AceriaDbContext).Assembly);
        }
    }
}

```

CSHARP

La primera línea declara el espacio de nombres de la infraestructura. La segunda declara el DbContext. La tercera declara el DbSet. La cuarta declara el constructor. La quinta sobrescribe OnModelCreating. La sexta aplica todas las configuraciones del ensamblado de infraestructura.

#### Configuraciones separadas por entidad

Las configuraciones de Fluent API se separan en clases que implementan IEntityConfiguration<T>. Cada clase configura una entidad. Esto mantiene el DbContext limpio y las configuraciones organizadas.

```
namespace AceriaData.Infrastructure.Persistence.Configurations
{
    public class OrdenFabricacionConfiguration : IEntityTypeConfiguration<OrdenFabricacion>
    {
        public void Configure(EntityTypeBuilder<OrdenFabricacion> builder)
        {
            builder.ToTable("OrdenesFabricacion");
            builder.HasKey(o => o.Id);

            builder.Property(o => o.NumeroOrden)
                .IsRequired()
                .HasMaxLength(50);

            builder.HasIndex(o => o.NumeroOrden)
                .IsUnique();
        }
    }
}
```

La primera línea declara el espacio de nombres. La segunda declara la clase que implementa `IEntityTypeConfiguration<OrdenFabricacion>`. La tercera declara el método `Configure`. La cuarta establece el nombre de la tabla. La quinta declara la clave primaria. La sexta selecciona la propiedad `NumeroOrden`. La séptima la marca como requerida. La octava establece la longitud máxima. La novena crea un índice. La décima lo marca como único. La configuración está aislada en su propia clase.

El proyecto `AceriaData` en capas

El proyecto `AceriaData` se refactoriza en cuatro proyectos: `AceriaData.Domain`, `AceriaData.Application`, `AceriaData.Infrastructure` y `AceriaData.Console`. `AceriaData.Domain` contiene las entidades del dominio. `AceriaData.Application` contiene los casos de uso y los puertos de persistencia, concretamente `IOrdenRepositorio` e `IUnidadDeTrabajo`. `AceriaData.Infrastructure` contiene el `DbContext`, las configuraciones, las migraciones y los adaptadores que implementan esos puertos. `AceriaData.Console` contiene el punto de entrada y actúa como composition root para registrar la infraestructura y ejecutar los casos de uso.

## Resumen de la teoría

La arquitectura limpia organiza el código en capas concéntricas.

La regla de dependencia establece que las dependencias apuntan hacia dentro.

La Arquitectura Hexagonal separa el núcleo de los adaptadores.

Los puertos son interfaces que expresan lo que el núcleo necesita.

Los adaptadores son las implementaciones concretas de los puertos.

`Domain` contiene las entidades y las reglas del dominio.

`Application` contiene los casos de uso y los puertos de persistencia, incluidos `IOrdenRepositorio` e `IUnidadDeTrabajo`.

La infraestructura contiene el `DbContext`, las configuraciones y las implementaciones de repositorios.

La capa de entrada contiene el método `Main` o los endpoints; en `AceriaData`, `Console` actúa también como composition root.

EF Core se coloca en la infraestructura.

La configuración de Fluent API se separa en clases por entidad.

El proyecto `AceriaData` se separa en cuatro proyectos.